Submission - Assignment 1

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Repositories

- Project repository: https://github.com/chechmek/EDPO_FS26_Project
- Exercise 1 repository: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests

1. General Project Description

1.1 Project Purpose and Value Proposition

The project is a peer-based content verification platform implemented as a set of Python microservices. Its core purpose is to address a trust problem that many digital platforms face: content should not become authoritative merely because it was published, but neither should trust decisions be delegated blindly to opaque automated scoring. Our project therefore proposes a middle ground between fully manual moderation and fully automated platform logic. Trustworthiness is established through an explicit, auditable process in which a registered user submits content, peers review it, the platform performs additional internal checks, and a cryptographic attestation is created only if the full process reaches a positive outcome.

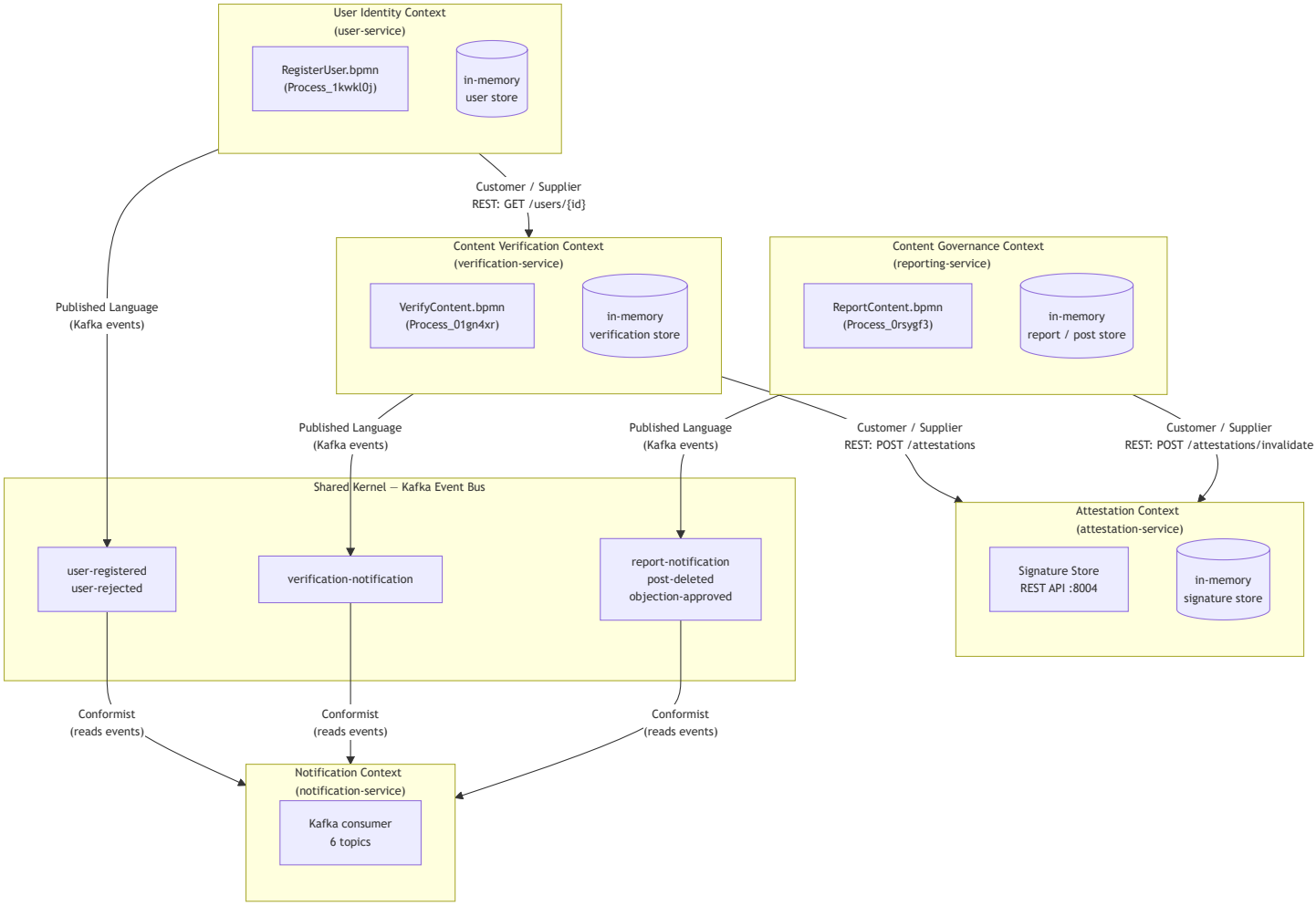
This makes the project more than a CRUD-style prototype. It is intentionally designed as a workflow-centric system in which trust, moderation, and accountability are first-class concerns. A content verification is not just a database update; it is a long-running decision process with waiting states, deadlines, external inputs, and legally or reputationally significant outcomes. The same is true for reporting and content removal. By modelling these flows explicitly, the platform can explain why a user was accepted, why content was verified, why a post was deleted, or why an objection was rejected. That proposition reflects the broader motivation of the course: using event-driven and process-oriented architecture not only to connect services, but to make distributed business behaviour explicit, observable, and governable.

At the business level, the platform offers three connected capabilities. First, it controls who may participate through a moderated onboarding process. Second, it creates verifiable trust claims for content through peer-based review and attestation. Third, it provides a governance process for contesting and removing problematic content. Together, these capabilities form a coherent lifecycle from identity, to verification, to moderation. The architectural proposition is that this lifecycle is best implemented through a hybrid design: BPMN orchestration for stateful business flows, and Kafka-based eventing for loosely coupled side effects such as notifications.

1.2 Domain Structure and Bounded Contexts

The system is organised into distinct bounded contexts rather than one monolithic application. This is important both conceptually and technically. The user domain is responsible for onboarding and registration status, the verification domain is responsible for peer-based trust decisions, the reporting domain governs moderation and deletion, the attestation domain stores signatures, and the notification domain reacts to published events. This separation allows each part of the platform to retain a clear responsibility while still participating in end-to-end processes.

The context map below shows how these domains relate to each other. `User Identity` is upstream of `Content Verification`, because verification depends on registration status. `Attestation` is a reusable capability consumed by both verification and governance. Kafka acts as a shared event bus for notification-style communication, and the `notification-service` behaves as a conformist consumer of the published event language.

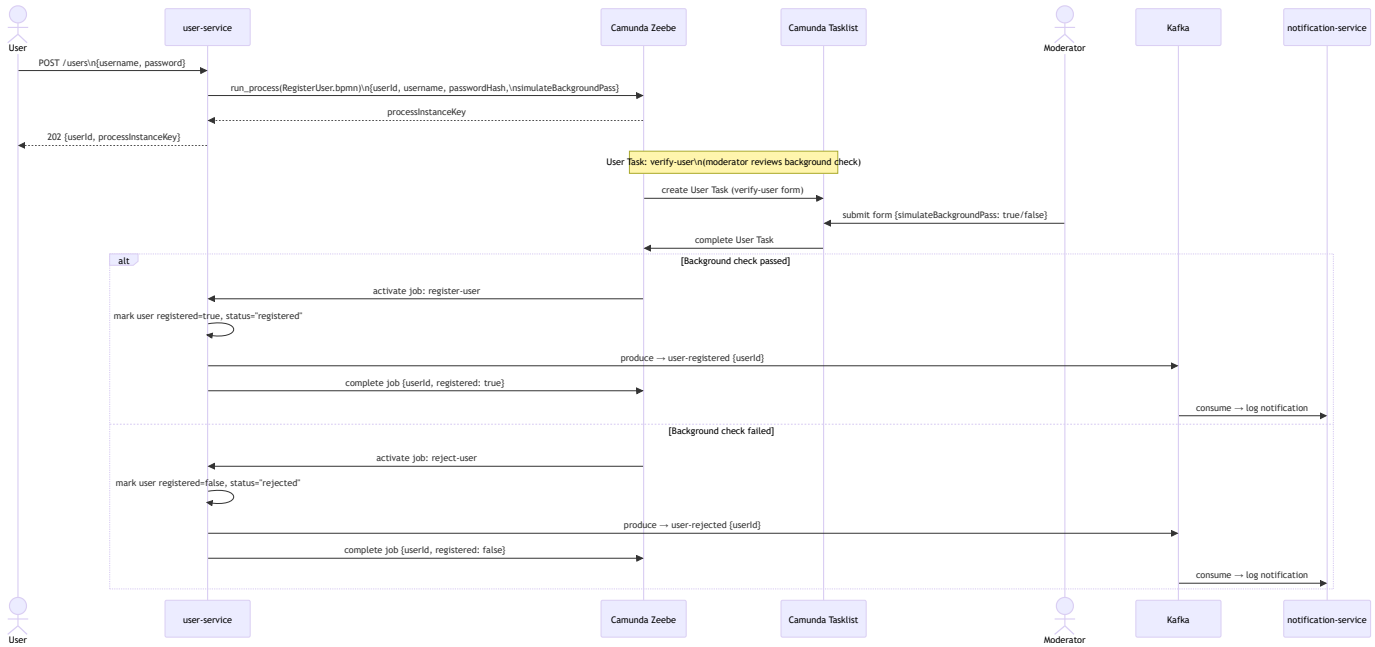


This decomposition is one of the key reasons the project remains understandable despite spanning several distributed concerns. Each process belongs to one service, and each service owns its own local state and task types. At the same time, the boundaries are deliberately porous through carefully chosen integration styles: synchronous REST where an immediate answer is required, Zeebe orchestration where explicit process state is needed, and Kafka where decoupled event distribution is preferable.

1.3 RegisterUser Process

`RegisterUser.bpmn` (`Process_1kwkl0j`) manages user onboarding (see Appendix D for the visual BPMN representation). A registration request is not automatically accepted, because the platform assumes that participation in later trust-related processes should itself be controlled. The flow therefore includes a human-reviewed background check in Camunda Tasklist before the user account is activated. Only after this decision does the service publish either a `user-registered` or a `user-rejected` event to Kafka. This makes onboarding part of the auditable lifecycle of the platform rather than a one-time setup step.

The sequence below shows that registration is both an orchestration problem and a governance problem. The process begins synchronously through the REST API, becomes stateful inside Zeebe, involves a human task, and only then emits an event for downstream consumers.

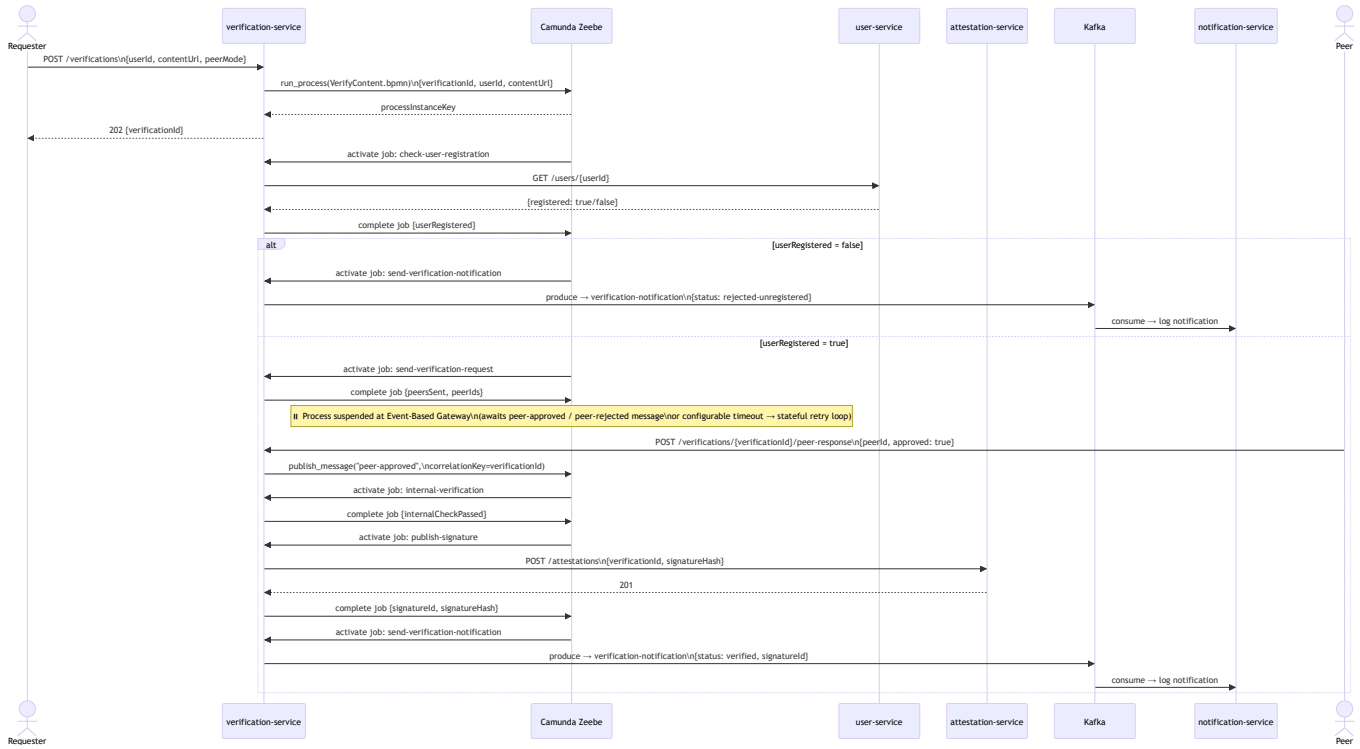


From a technical perspective, the process already demonstrates the overall style of the platform. The `user-service` exposes the external API, starts the BPMN process, executes the service tasks through a Zeebe worker, and persists its domain-facing state in an in-memory store. The generated `userId` then becomes the stable identifier used by the verification flow. In other words, this onboarding process does not stand alone; it establishes the identity foundation on which later trust decisions depend.

1.4 VerifyContent Process

`VerifyContent.bpmn` (`Process_01gn4xr`) governs the full content verification lifecycle and is the central value-creation process of the platform (see Appendix D for the visual BPMN representation). After a registered user submits a content URL, the platform checks eligibility, dispatches peer review requests, waits for peer verdicts through message correlation, performs an internal verification step, and, if the outcome is positive, stores a cryptographic signature through the `attestation-service` . The proposition of the system is concentrated here: trust is not inferred silently in the background, but earned through a transparent, multi-step workflow that can later be inspected and explained.

This is also the most technically expressive process in the project because it combines several EDPO concepts in one flow: synchronous REST to validate a dependency on the `user-service` , asynchronous waiting via an Event-Based Gateway, stateful retry when peer responses are delayed, internal worker-driven service tasks, a REST-based infrastructure call to the `attestation-service` , and Kafka publication of the final result.

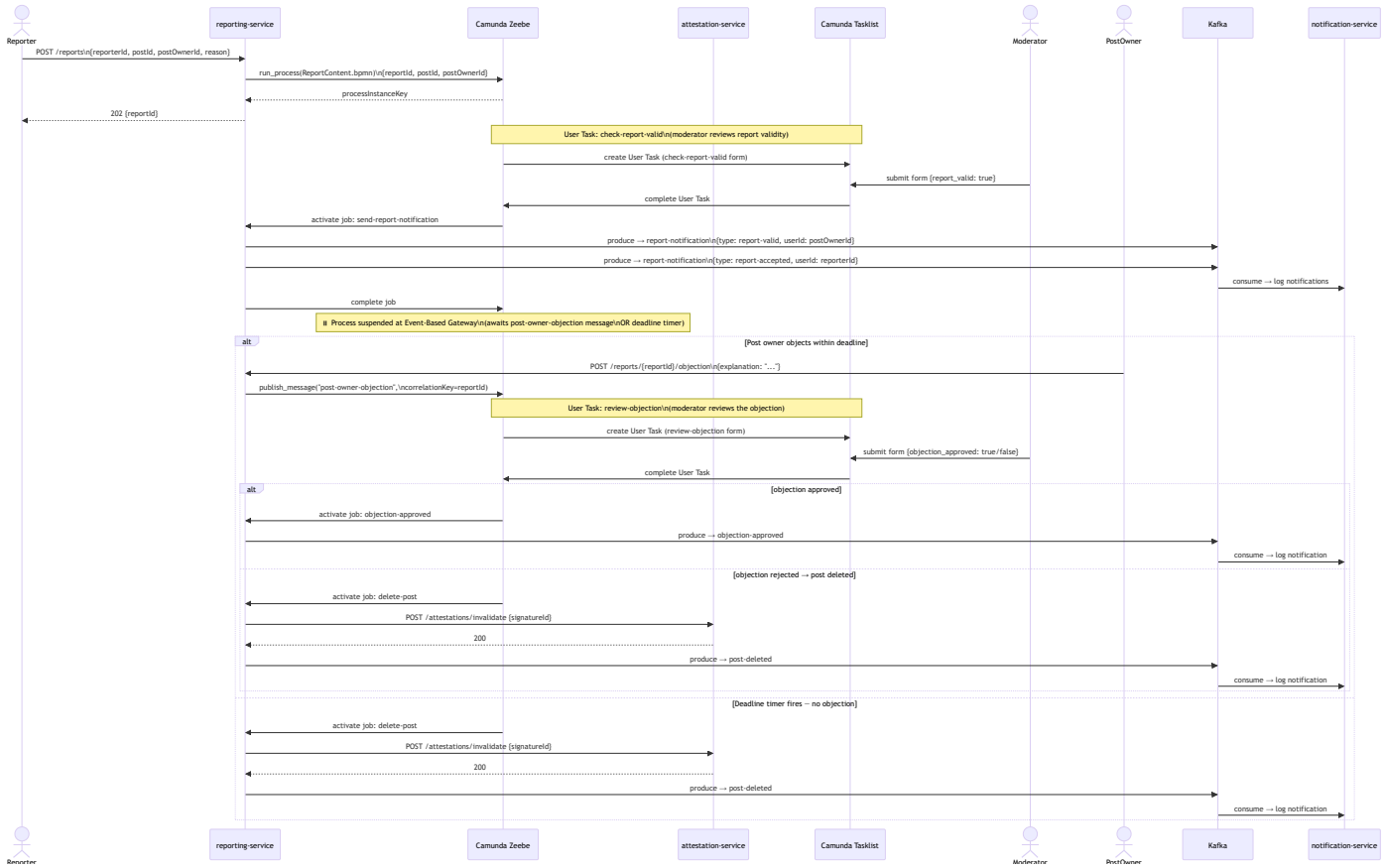


Technically, the `verification-service` is the most orchestration-heavy service in the system. It combines API handling, Zeebe process starts, worker execution, HTTP calls to other services, and Kafka publishing. Its central process variable `verificationId` is both a client-visible identifier and the correlation key for Zeebe messages, which keeps the API contract and the workflow state aligned. The process also demonstrates why the platform uses Camunda 8 rather than plain event choreography: peer review is long-running, externally driven, and failure-sensitive, so the system needs explicit process state rather than a chain of stateless consumers.

1.5 ReportContent Process

`ReportContent.bpmn` (`Process_0rsygf3`) covers the moderation and governance dimension of the platform (see Appendix D for the visual BPMN representation). A reporter can flag a post as problematic, but deletion is not executed as an immediate automated reaction. Instead, the process starts with moderator validation, informs the affected post owner, opens a formal objection window, and handles the outcome depending on whether an objection is filed and how it is reviewed. This process therefore gives the platform procedural fairness: it makes moderation not only effective, but explainable and contestable.

The flow is especially important from an architectural point of view because it combines human judgement, time-based waiting, asynchronous callbacks, and compensating action on related infrastructure state. If the post is ultimately deleted, the process also invalidates the attestation signature through the `attestation-service` , which ties governance back to the platform's trust model.



In technical terms, the `reporting-service` demonstrates the strongest human-in-the-loop characteristics of the platform. It starts the process, participates as Zeebe worker, correlates external objection messages back into the workflow, calls the `attestation-service` for signature invalidation, and publishes all externally visible outcomes to Kafka. The process is intentionally designed so that irreversible actions do not happen before the possibility of review and objection has been exhausted.

1.6 Technical Architecture and Runtime Setup

The entire stack is containerised with Docker Compose, but the runtime architecture is intentionally split between infrastructure concerns. Camunda 8 Run (`c8run`) operates on the host, exposing Zeebe gRPC on port 26500 and Camunda Operate and Tasklist on port 8080, while the application services and Kafka infrastructure run in containers. This split reflects a practical development trade-off: it keeps the local Zeebe runtime easy to inspect and manage while still allowing the services to be deployed and rebuilt consistently through Docker.

From a communication perspective, the project combines three integration styles. First, HTTP/REST is used for user-facing APIs and for selected synchronous service-to-service checks, such as `verification-service` calling `user-service` or either domain service calling `attestation-service` . Second, Zeebe gRPC is used for process starts, job activation, job completion, and message publication. Third, Kafka is used for notification-style events that should remain decoupled from the success of the core flow. This gives each communication channel a clear semantic role rather than mixing all interactions through a single protocol.

The five services are implemented in Python 3.12 with Flask. The three orchestrated services connect to Zeebe through `pyzeebe` , an asyncio-native Python library that supports process execution and job-worker behaviour. Each of them runs a Zeebe worker in a dedicated background loop and exposes an HTTP API for starting or interacting with process instances. The `attestation-service` is deliberately simpler: it is a REST-only service that stores and invalidates signatures. The `notification-service` has the opposite profile: it exposes no HTTP interface at all and exists purely as a Kafka consumer that materialises user-facing notification side effects through log output.

At the current project scale, all service-local state is stored in memory, which keeps the implementation lightweight and easy to understand for a course project. That decision also makes some limitations explicit and therefore educational: a service restart clears

local read-side state, while the authoritative workflow state remains in Zeebe. This contrast highlights the difference between local service memory and durable process state, which is one of the central architectural ideas explored throughout the project.

2. Concepts from the Lecture and Exercises

2.1 Event-Driven Architecture and Streaming Fundamentals

The first lectures introduced Kafka not merely as a queue, but as a distributed event streaming platform built around durable logs, topics, partitions, producer and consumer roles, and consumer-group based scalability. That distinction matters directly for this project. We use Kafka only where the lecture material suggests that loose coupling and replayability are more valuable than end-to-end process control: notification-style side effects. In concrete terms, `user-service`, `verification-service`, and `reporting-service` publish events such as `user-registered`, `verification-notification`, `report-notification`, `post-deleted`, and `objection-approved`, while `notification-service` consumes them independently. This design follows the Event Notification pattern discussed in the early weeks and later justified formally in ADR-0005, where Kafka was chosen over a traditional MQ because retained logs and independent consumer groups fit our use case better than broker-managed push delivery.

The choice is not just conceptual; it is reflected in the concrete producer and consumer configuration. Kafka lectures emphasised that offsets, partitions, and consumer groups are what make scalability and fault tolerance possible. Our `notification-service` therefore uses a named consumer group and manual offset commits, so recovery semantics remain under application control instead of being delegated to auto-commit behaviour. Likewise, producer settings such as `acks=all` and idempotence reflect a deliberate trade-off between latency and reliability: each notification incurs slightly more overhead, but the producer is resilient under retries and future replication scenarios. The resulting architecture covers the material from Lecture 1 on topics, partitions, offsets, and consumer groups, while the implementation experience from the early exercises showed us that the broker becomes much more than a transport layer: it also becomes a debugging and observability tool, especially through Kafka UI.

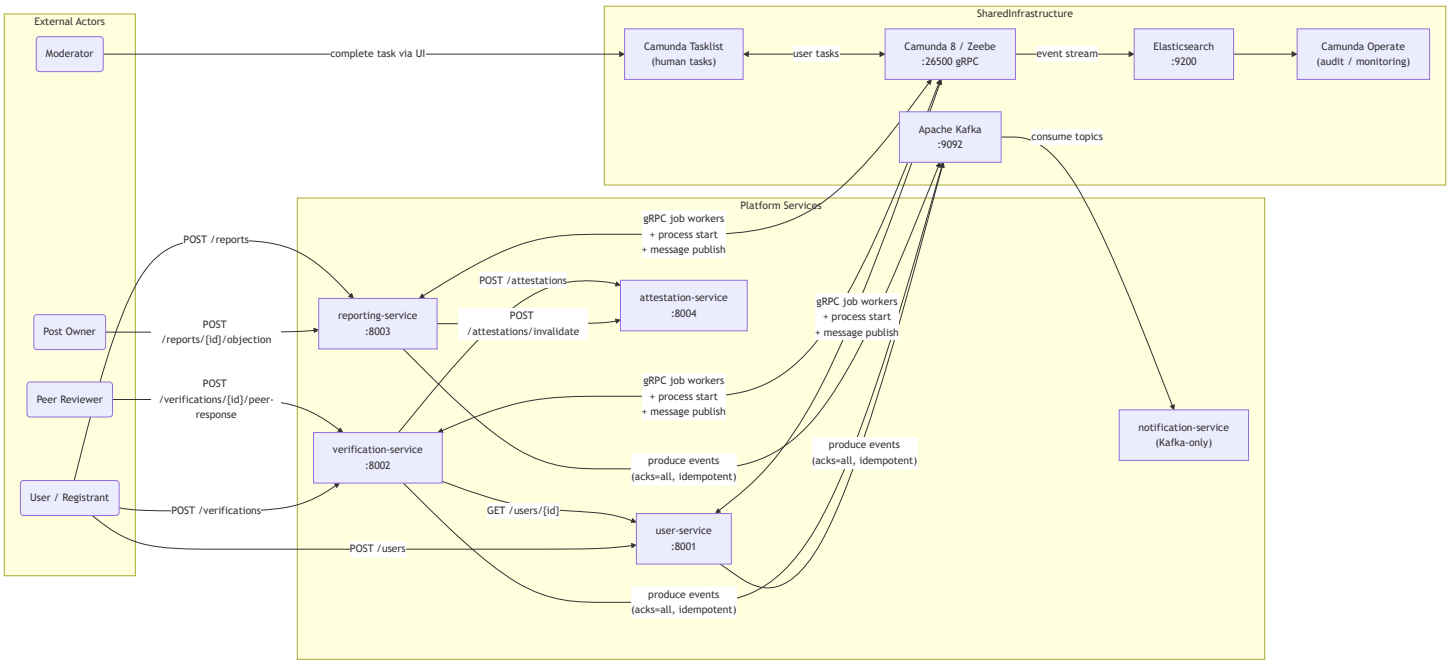
At the same time, the project deliberately does not use Kafka for everything. One of the key takeaways from the course is that event-driven decoupling is powerful, but not automatically the right choice for long-running, stateful business flows. That negative decision is just as important as the positive one. We use Kafka where replayable, fan-out style side effects are desirable, but we avoid encoding the core verification and moderation logic as a chain of opaque topic subscriptions because that would distribute control flow and error handling across multiple services.

2.2 Process-Oriented Architecture, Orchestration, and Workflow Ownership

From Lecture 3 onward, the course moved from event streams to process automation and asked when choreography stops being sufficient. That tension is central to our project. User onboarding, content verification, and reporting all involve explicit waits, branching decisions, deadlines, and legally or reputationally significant outcomes. In the lecture's terms, these are precisely the situations in which a process owner should exist. Our system therefore models `RegisterUser.bpmn`, `VerifyContent.bpmn`, and `ReportContent.bpmn` as first-class BPMN processes executed on Camunda 8 (Zeebe), rather than leaving the end-to-end lifecycle implicit across a set of event handlers.

This design directly reflects the argument developed in the orchestration lectures: choreography maximises decoupling and should be the default for distributed systems, but it loses visibility when the workflow itself becomes a business asset. The project therefore adopts the hybrid coordination strategy documented in ADR-0001: orchestration for stateful, conditional, auditable business flows; choreography for notifications and other non-blocking side effects. This is not a compromise made out of indecision, but a rule-based allocation of responsibilities. The orchestrator is used as an explicit process owner, not as a "god service". Domain logic remains in the services; the engine coordinates steps, deadlines, and state transitions. That distinction mirrors the lecture's warning that orchestration does not create poor architecture by itself; poor service boundaries do.

The following system-context diagram should therefore be read not as an isolated illustration, but as a condensed architectural argument. It shows that the user-facing and peer-facing interactions enter the system through domain services, while Zeebe, Kafka, Tasklist, and Operate each support a distinct coordination or observability concern.



The BPMN implementation itself covers a broad set of process concepts from the middle part of the course: service tasks with explicit worker types, user tasks with forms, event-based gateways, exclusive gateways, timer events, message catch events, and process-level ownership. In implementation terms, this is where ADR-0002 becomes relevant. Camunda 8 was chosen over Camunda 7 because the project is written in Python and `pyzeebe` provides a clean, native integration path. The decision was therefore not only architectural but socio-technical: it aligned the process engine with the team's language, deployment model, and operational capabilities.

2.3 Service Boundaries, Bounded Contexts, and the Avoidance of a Process Monolith

One of the recurring ideas in both the lectures and the later exercises is that process automation only remains maintainable if the boundaries of the automated process align with meaningful domain boundaries. We therefore did not model one giant BPMN process that spans registration, verification, and reporting. Instead, as formalised in ADR-0006, the system is split into three bounded contexts with one BPMN model each: user identity, content verification, and content governance. This reflects the DDD-oriented way of thinking that was repeatedly encouraged in class, even when the explicit terminology was not always the primary focus of a given lecture.

This decision carries several architectural benefits. It keeps model ownership local to the corresponding service, isolates change, and prevents one busy or frequently modified process from becoming a bottleneck for unrelated flows. The lectures on orchestration repeatedly warned about the danger of rebuilding centralised monoliths in a new form. Our answer to that risk is not to reject orchestration altogether, but to constrain it with service boundaries. Each service owns exactly the job types in its own BPMN model and no service becomes a generic executor for someone else's workflow. The `attestation-service` is intentionally not modelled as its own process because it does not represent a business workflow; it represents a reusable infrastructure capability. That distinction between domain process and technical dependency is subtle but important, and it helped keep the orchestration layer lean.

2.4 Stateful Resilience, Event-Sourced Workflow State, and Human Intervention

Lecture 6, together with Exercise 5 and the Flowing Retail material, made the strongest case for workflow engines as state managers rather than just task routers. That idea is directly realised in our project. `VerifyContent.bpmn` and `ReportContent.bpmn` are both long-running processes whose correctness depends on state surviving crashes, redeployments, and delayed external input. Zeebe's event-sourced execution model is therefore not an implementation detail; it is the enabling property behind our resilience strategy. Instead of

writing explicit checkpointing logic in each service, we rely on the engine to persist the process state as an append-only log and reconstruct it from snapshots, which is the core point of ADR-0003.

The stateful retry pattern is implemented in `VerifyContent.bpmn` exactly because a peer-review request can fail in ways that are not catastrophic but still require time-aware recovery. The retry counter `send_verify_retry` is not an in-memory variable in the worker; it is a process variable under engine control. This means that timeout handling, retry budgeting, and eventual failure notification remain correct even if the worker disappears during the waiting period. That is the practical form of the distinction discussed in class between transient worker-local retries and stateful workflow-managed retries.

Human intervention is equally important. The course material repeatedly emphasised that some failures should not trigger blind automation, especially when legal, social, or reputational effects are involved. Our reporting flow therefore pauses for moderator action at `check-report-valid` and `review-objection`, and the registration flow uses `verify-user` as a human gate before activation. This is not only a resilience pattern but also a governance mechanism. Human tasks make the exceptions explicit and auditable rather than hidden in application logs or admin scripts.

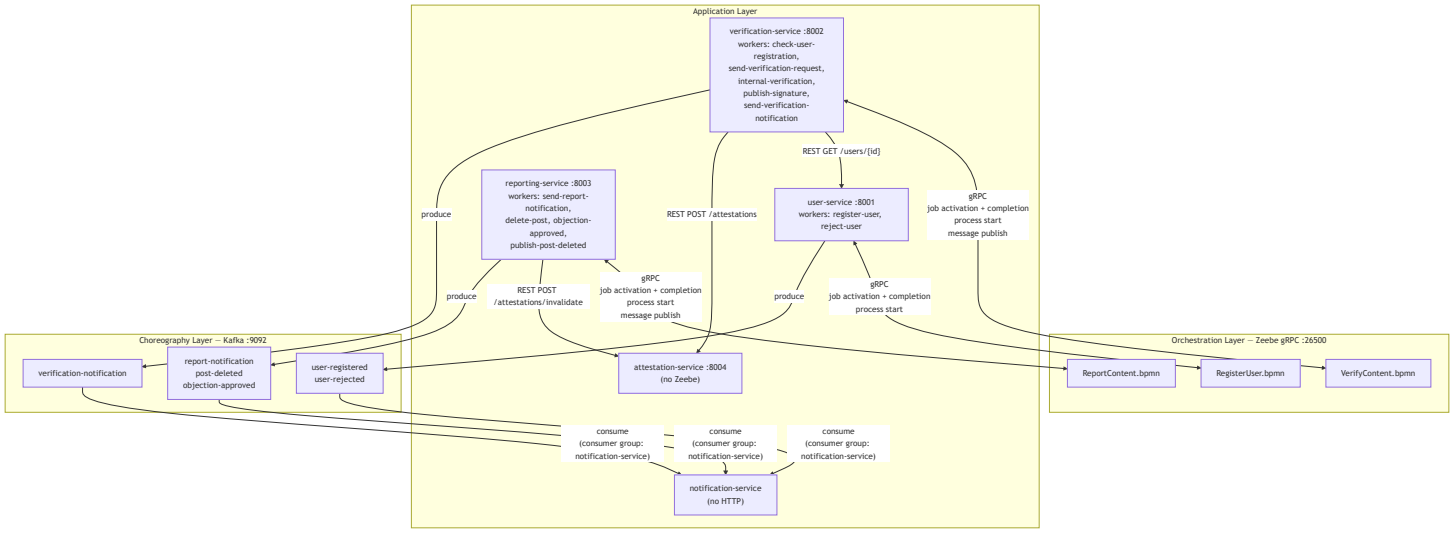
2.5 Sagas, Eventual Consistency, and Asynchronous Correlation

Lecture 5 framed distributed consistency as a choice among saga styles and consistency models rather than as an attempt to recreate monolithic ACID behaviour across service boundaries. That framing influenced our architecture strongly. As captured in ADR-0004, each service owns its own state and cross-service coordination happens through orchestration and messaging rather than shared storage. In this sense, `VerifyContent` behaves like an orchestrated saga with eventual consistency: user registration is checked synchronously through `user-service`, attestation is delegated to `attestation-service`, and failure paths are handled explicitly by the workflow rather than being hidden behind rollback semantics that do not exist in a distributed system.

This is also where the trade-offs from the saga lecture become concrete. We consciously accept that local service views can temporarily diverge. For example, attestation may already have been stored while a service-local in-memory representation has not yet been updated, or a restarted service may temporarily lose its read-side cache even though the process state remains valid in Zeebe. The lecture's point that eventual consistency is not "free" but requires explicit workflow state management is visible here in practice. The benefit is responsiveness and service autonomy; the cost is that consistency is achieved through process progression rather than cross-service transactions.

The second concept that is central here is asynchronous message correlation, formalised in ADR-0007. Instead of polling for peer verdicts or objections, the workflows suspend at message catch events and are resumed when the relevant REST endpoint publishes a correlated Zeebe message. The user-visible `verificationId` and `reportId` are therefore not only API identifiers but also workflow correlation keys. This is a particularly strong example of course concepts influencing the concrete API design: the correlation key is chosen once and then reused consistently across process state, external callback contract, and engine routing semantics.

The following service-level diagram illustrates how these concepts come together operationally. It should be read as a map of coordination styles: Zeebe for orchestrated state transitions, REST for direct synchronous dependency checks, and Kafka for non-blocking publication of side effects.



In the terminology of the saga lecture, our system sits closest to an orchestrated, eventually consistent style rather than an atomic distributed transaction model. It also demonstrates the complementary relationship between synchronous and asynchronous communication: synchronous REST is used when a task handler needs an immediate answer in order to advance the process, while asynchronous messaging is used when the process can safely wait or when a side effect should not block completion.

2.6 CQRS, Auditability, and Operational Visibility

The later part of Lecture 6 broadened the perspective from process execution to process observability, and this influenced one of the most important decisions in the project. The platform makes decisions that must be explainable afterwards: a user can be rejected, content can be certified, a report can lead to deletion, and a delayed objection can be ignored because the deadline expired. For such a system, the write side alone is not enough. We therefore use Camunda Operate and Elasticsearch as a CQRS-style read side, as summarised in ADR-0008.

This decision matters conceptually because it shows that auditability is not an afterthought layered on top of service code. Instead, the workflow engine emits the process event stream, Operate projects it into a queryable model, and Tasklist exposes the human-work interface that complements the automated flow. This covers course topics far beyond simple BPMN execution: event sourcing, read/write model separation, process monitoring, human task management, and the operational consequences of projection lag and externalised read models. The trade-off is familiar from the lecture material: Operate and Elasticsearch introduce additional operational weight, but they remove the need to build a bespoke audit database and provide far stronger diagnostic capabilities than application logs alone.

Together, these concepts show why the project is not just a BPMN wrapper around REST calls. It is a deliberate combination of event streaming, workflow orchestration, stateful resilience, saga-style coordination, asynchronous message correlation, CQRS, and bounded-context-oriented decomposition. The resulting architecture is broader than any single weekly topic and mirrors the structure of the course itself: event-driven mechanisms where loose coupling is beneficial, process-oriented control where explicit state and accountability are required, and careful trade-off decisions where the two approaches meet.

3. Results and Insights

3.1 Kafka

Configuring producers with `acks=all` in a single-broker development setup means each produce call waits for the broker to acknowledge the write. This adds a small but perceptible latency to each notification event. In a multi-broker cluster with in-sync replicas, this configuration provides meaningful durability guarantees; in a single-broker setup it is equivalent to `acks=1` in practice but prepares

the code for production conditions. Combined with `enable.idempotence=True`, this configuration ensures that no notification event is lost or duplicated under retries.

The decision to use manual offset commits in the `notification-service` rather than the default auto-commit proved important during testing. With auto-commit enabled, a crash between receiving a message and processing it would silently drop the message because the offset had already advanced. Manual commit after the handler ensures at-least-once processing semantics at the consumer side.

The Kafka UI at `localhost:8079` was used extensively throughout development to inspect message payloads, verify that events were being produced on the correct topics, and diagnose consumer group lag. This level of visibility would have been significantly harder to achieve with a traditional message queue.

3.2 Camunda and Zeebe

One of the more counterintuitive lessons from working with Zeebe was the behaviour of in-memory service state after a restart. The `_verifications` and `_reports` dictionaries in the application services are populated when a process is started via the REST API. If a service container restarts, those dictionaries are empty and any subsequent `GET` request for a running verification or report returns 404, even though the process instance is still alive in Zeebe. This is an accepted limitation at the current project scale and is documented in ADR-0003, but it was initially surprising and underscores the importance of treating Zeebe as the authoritative state store rather than application-tier memory.

Zeebe's job activation timeout provided an implicit crash recovery mechanism that we had not planned for explicitly. When a service container restarted mid-job, Zeebe simply waited for the activation timeout to elapse and then re-offered the job to the next available worker. No application code was needed to detect or recover from this situation.

The Message TTL behaviour for late peer verdicts also required attention. If a peer submits a verdict after the verification process has already timed out and completed, the published Zeebe message finds no active subscription and is silently discarded after the TTL. This is the correct behaviour — a verdict for a closed process should not reopen it — but it means callers receive no error from the REST endpoint. If explicit feedback to late-arriving peers were required, an application-layer check on the process state would be needed before publishing the Zeebe message.

Running Camunda 8 Run (`c8run`) on the host rather than in Docker introduced a Java 21 dependency that falls outside the Python ecosystem. Every developer needed a working Java installation alongside Docker. This friction was the primary operational cost of choosing Camunda 8 over lighter alternatives.

3.3 Impact of Service Unavailability

The hybrid architecture produced clearly differentiated resilience characteristics depending on which component was unavailable.

If the `notification-service` is stopped, all six orchestrated business flows continue without interruption. Kafka retains the events on disk; when the notification service restarts and rejoins its consumer group, it reads from its last committed offset and processes all messages it missed. This is the core extensibility property that justified Kafka's operational overhead over a simpler queue.

If a `ZeebeWorker`-bearing service container crashes mid-job, Zeebe's activation timeout automatically returns the job to the available queue. When the container restarts and reconnects, its worker picks up the job and continues. From the perspective of the process instance, the crash is invisible; from the perspective of the application developer, no recovery code was required.

If Zeebe itself is unavailable, no new process instances can start and no in-flight jobs can progress. This is the acknowledged single point of failure for the current single-broker setup. A production deployment would require a multi-broker, multi-partition Zeebe cluster and a health-checked deployment pipeline to mitigate this risk.

4. Reflections and Lessons Learned

The most significant architectural lesson from this project is that BPMN makes business logic explicit in a way that code alone does not. Every timeout, every retry boundary, every conditional branch, and every human decision point in the content verification and reporting flows is directly readable in the BPMN model — no need to trace through service code to understand what happens when a peer is slow to respond or when a moderator approves an objection. This transparency proved valuable not only for design but for debugging: when a process instance behaved unexpectedly, opening it in Camunda Operate and reading its variable history felt rather easier than adding lots of logging statements and tracing it that way.

The hybrid coordination approach — Zeebe for stateful orchestration, Kafka for notifications — held up well throughout the project. The right-tool-for-the-right-job principle avoided the two failure modes we had anticipated: a process monolith where even simple notifications were modelled as BPMN tasks, and a pure choreography system where the verification flow's conditional branching and compensation logic would have been scattered across event handlers in multiple services.

`pyzeebe` made Zeebe genuinely accessible from Python. Writing a new job worker is a matter of defining an `async def` function and decorating it with `@worker.task(task_type="...")`. The library abstracts away gRPC internals, event loop management, and reconnection logic entirely. Without it, integrating a Python service with Zeebe would have required a bespoke polling loop and serialization layer.

Working with in-memory service state highlighted a real production concern. At course scale the accepted limitation — that service state is lost on restart — is manageable. In a real deployment, each service would require a persistent backing store to ensure that GET endpoints remain consistent with the Zeebe process variables that represent the authoritative state. The database-per-service model is already enforced architecturally; adding persistence would be a matter of wiring in a database client per service without structural changes.

Kafka's operational overhead — the need to understand consumer group offsets, topic configurations, and broker settings — was the highest onboarding cost of the project. However, the extensibility benefit paid off concretely: the `notification-service` was added after the other four services were working, and it required zero changes to any existing producer. Adding a hypothetical analytics service would require the same effort: create a new consumer group, subscribe to the relevant topics, deploy. That additive quality makes the overhead worthwhile.

Finally, the at-least-once delivery commitment — committing domain state before completing the Zeebe job — requires that all job workers be designed for idempotent re-execution. This is a discipline rather than a framework guarantee, and it requires deliberate thought when writing each handler: if this job runs twice, does the second execution leave the system in a consistent state? In practice this meant writing simple, side-effect-free checks before performing mutations, and ensuring that calls to the `attestation-service` and Kafka producers were safe to repeat.

5. GitHub Release

Assignment-1 release (project / E2-E5): https://github.com/chechmek/EDPO_FS26_Project/releases/tag/assignment-1

Further information provided in the repository README.md.

Assignment-1 release (Kafka-Tests / E1): https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/releases/tag/assignment-1

Further information provided in the repository README.md.

Appendix

The following documents are attached to this submission PDF as appendices. They are maintained as separate files in the repository and are included here without modification.

Appendix A — Exercise 5 Report: Stateful Resilience Patterns
Detailed report on the implementation of Stateful Retry and Human Intervention in the BPMN processes, including process-level design rationale and lecture references.
File: submission-exercise-5.md

Appendix B — Team Responsibilities
Full exercise-by-exercise responsibility table with links to the repository and commit history.
File: submission-responsibilities.md

Appendix C — Architecture Diagrams
Complete set of six conceptual diagrams: System Context, DDD Context Map, Service Architecture, and three end-to-end sequence diagrams (RegisterUser, VerifyContent, ReportContent).
File: submission-diagrams.md

Appendix D — BPMN Models and Forms
The complete executable process models and their associated Camunda forms used in the project. These files document the workflow logic directly at the modelling level and complement the textual and diagrammatic explanations in the main submission.

File: submission-bpmn-files.md

Appendix E — Architecture Decision Records
Full text of all eight ADRs covering coordination pattern, workflow engine selection, stateful resilience, distributed data consistency, message broker selection, process model boundaries, asynchronous message correlation, and auditing.

ADR	Title	Main Contribution to the Architecture	Key Trade-off
ADR-0001	Coordination Pattern: Hybrid Orchestration and Choreography	Establishes the hybrid model of Zeebe orchestration plus Kafka choreography	Better visibility and control for core flows at the cost of a larger operational surface
ADR-0002	Workflow Engine Selection: Camunda 8 over Camunda 7	Chooses Camunda 8 because Python services can use `pyzeebe` and benefit from event-sourced state	Stronger workflow support and crash resilience at the cost of Java-based infrastructure
ADR-0003	Stateful Resilience and Error Handling	Puts retries and crash recovery into Zeebe and routes irreversible cases to human tasks	Simpler services and clearer recovery paths, but a hard dependency on Zeebe and Tasklist
ADR-0004	Distributed Data Consistency: Saga Pattern over Distributed ACID Transactions	Enforces database-per-service and saga-style coordination instead of distributed transactions	Independent services and scalable coordination, but temporary inconsistency and explicit compensation
ADR-0005	Message Broker Selection: Apache Kafka for Choreography Events	Picks Kafka for retained, replayable, multi-consumer notification streams	More extensibility and durability, but more operational complexity than a simple queue

ADR	Title	Main Contribution to the Architecture	Key Trade-off
ADR-0006	Process Model Boundaries: One BPMN Model per Bounded Context	Prevents a process monolith by aligning one BPMN model with one bounded context	Better ownership and isolation, but cross-context queries require aggregation
ADR-0007	Asynchronous Message Correlation via Zeebe	Uses native Zeebe message correlation for peer verdicts and objections	No polling and no routing tables, but late messages may be discarded after TTL
ADR-0008	Auditing and Read Models: Camunda Operate as a Separated CQRS Read Side	Uses Operate and Elasticsearch as the audit-oriented read side	Strong observability without custom code, but additional memory and infrastructure overhead

Appendix A - Exercise 5

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Repository

- GitHub repository: https://github.com/chechmek/EDPO_FS26_Project

Team Contribution

- Evan Martino worked on the implementation and deployment of the processes.
- Roman Babukh worked on the implementation of the non-process-related parts of the project.
- Marco Birchler worked on the documentation and submission.

For an overview of responsibilities across all exercises, see `[submission-responsibilities.md](./submission-responsibilities.md)`.

Report on Stateful Resilience Patterns in the Software Project

For our content verification platform we implemented two of the stateful resilience patterns introduced in the lecture and Lab08 (Flowing Retail): **Stateful Retry** and **Human Intervention**. Both patterns are modelled directly in BPMN and executed by Camunda Zeebe. Because Zeebe is an **event-sourced workflow engine**, all changes to process state are recorded in an append-only event log and materialized into RocksDB snapshots (Lecture 6, slides 4-5). The current state of any process instance can be fully reconstructed from this log, meaning process variables survive crashes and redeployments without any additional persistence code on our side (Lecture 6, slide 8). This is the foundational property that makes stateful resilience patterns practical. Zeebe does not implement ACID transactions across services. As discussed in class, this presents two design options: commit to the domain first and then notify Zeebe (at-least-once), or notify Zeebe first and then commit (at-most-once) (Lecture 6, slide 25). We follow the **at-least-once** approach, committing domain state before completing the Zeebe job, accepting the possibility of duplicate processing in exchange for the guarantee that no state update is ever silently lost. Our patterns therefore rely on **state management and eventual consistency** rather than distributed transactions.

2.1 Stateful Retry: `VerifyContent.bpmn`

Implementation: In `VerifyContent.bpmn`, after `Send Verification Request to Peers` completes, an Event-Based Gateway waits for either a peer response message or a configurable timer. On timeout or service error, a Script Task increments `send_verify_retry` and an exclusive gateway routes back to retry or forward to a failure notification once the limit is reached.

Peer verification is the core value proposition of the platform: a cryptographic signature is only meaningful if it was issued after a genuine human review. Peers are humans who may be slow, offline, or simply unresponsive, so transient non-responses are expected and should not immediately cause a verification to fail.

As discussed in class, **stateful retry** differs fundamentally from a simple in-process retry: the retry state is owned by the workflow engine rather than the job worker. This matters here because the retry boundary spans an asynchronous wait that can last minutes. A job-worker retry would be reset by any crash during that window; a Zeebe process variable is not, because the engine reconstructs it from the event log on recovery. The retry count is also fully visible in **Camunda Operate** via the CQRS exporter to Elasticsearch, making the behaviour auditable (Lecture 6, slides 26-27). Capping retries at three prevents indefinite looping; after exhausting retries the requester receives a clear notification rather than a silent failure.

The main tradeoff is added process complexity: the retry loop, counter variable, and boundary error event all need to be modelled and tested explicitly. We accepted this cost because silently dropping a failed verification or failing on the first timeout would undermine trust in the platform.

2.2 Human Intervention: `ReportContent.bpmn`

Implementation: In `ReportContent.bpmn`, after the post owner is notified, an Event-Based Gateway waits for either an `Objection` message or a deadline timer. The objection message uses Zeebe's **message correlation** mechanism: a subscription is registered on a partition determined by a hash of `reportId`, so the arriving message is reliably routed back to the correct waiting process instance (Lecture 6, slide 24). If an objection arrives, a `Review Objection` User Task is created in Camunda Tasklist for a moderator, who approves or rejects it via a rendered form. If the deadline fires with no objection, the post is deleted automatically.

Content moderation is an inherently subjective judgement that carries legal and reputational risk. As covered under **human intervention** in class, there are decisions that should not be fully automated: the consequences are irreversible (deletion) and the inputs are ambiguous (a contested report). By suspending the process at an Event-Based Gateway and giving the post owner a fixed window to raise an objection, we provide due process before anything is deleted. If an objection arrives, a moderator makes the final call rather than the system. This matches the human-in-the-loop pattern from the lecture, where a User Task pauses automated flow until a human provides a decision.

The tradeoff is that the process may stay open for the full deadline period before acting, but this is an intentional and explainable delay rather than a system failure.

References

Technical References

- BPMN files: `[bpmn files/](../bpmn%20files/)`
- Service skeletons: `[services/](../services/)`
- Infrastructure: `[docker-compose.yml](../docker-compose.yml)`
- Exercise 3 submission (process design detail): `[submission-exercise-3.md](./submission-exercise-3.md)`
- Exercise 4 submission (orchestration & ADR): `[submission-exercise-4.md](./submission-exercise-4.md)`

Images of Processes and Forms

- VerifyContent process (stateful retry): `[images/E5/VerifyContent_E5.png](./images/E5/VerifyContent_E5.png)`
- ReportContent process (human intervention): `[images/E5/ReportContent_E5.png](./images/E5/ReportContent_E5.png)`
- RegisterUser process: `[images/E5/RegisterUser_E5.png](./images/E5/RegisterUser_E5.png)`
- Verify user: `[images/E5/VerifyUser_E5.png](./images/E5/VerifyUser_E5.png)`
- Check report validity: `[images/E5/CheckReportValidity_E5.png](./images/E5/CheckReportValidity_E5.png)`
- Review objection: `[images/E5/ReviewObjection_E5.png](./images/E5/ReviewObjection_E5.png)`

Appendix B - Team Responsibilities

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Further Information

- GitHub Repository: chechmek/EDPOFS26Project
- Commit History: github.com/chechmek/EDPOFS26Project/commits/main

Responsibilities

Exercise	Task	Responsible	Description
1	Experimenting	Evan	Producer-focused experiment runs and review on Evan
1	Experimenting	Marco	Experiment design and execution on Marco
1	Experimenting	Roman	Experiment framework and outputs on Roman
1	Revision	Roman	Revised parts of Exercise 1 according to feedback
2	Implementation	Roman	Focused primarily on the initial implementation of the project
2	Documentation	Marco	Worked on the documentation and submission
2	Support	Evan	Supported the other team members in both tasks
3	BPMN	Marco	Created the original BPMN processes
3	Improving	Evan	Refined the core concepts and BPMN processes
3	Documentation	Roman	Worked on the documentation and submission
4	Improving	Marco	Refined the core concepts and BPMN processes
4	Improving	Evan	Refined the core concepts and BPMN processes
4	Documentation	Roman	Worked on the documentation and submission
5	Implementation	Evan	Worked on the implementation and deployment of the processes
5	Implementation	Roman	Worked on the implementation of the non-process related parts of the project
5	Documentation	Marco	Worked on the documentation and submission

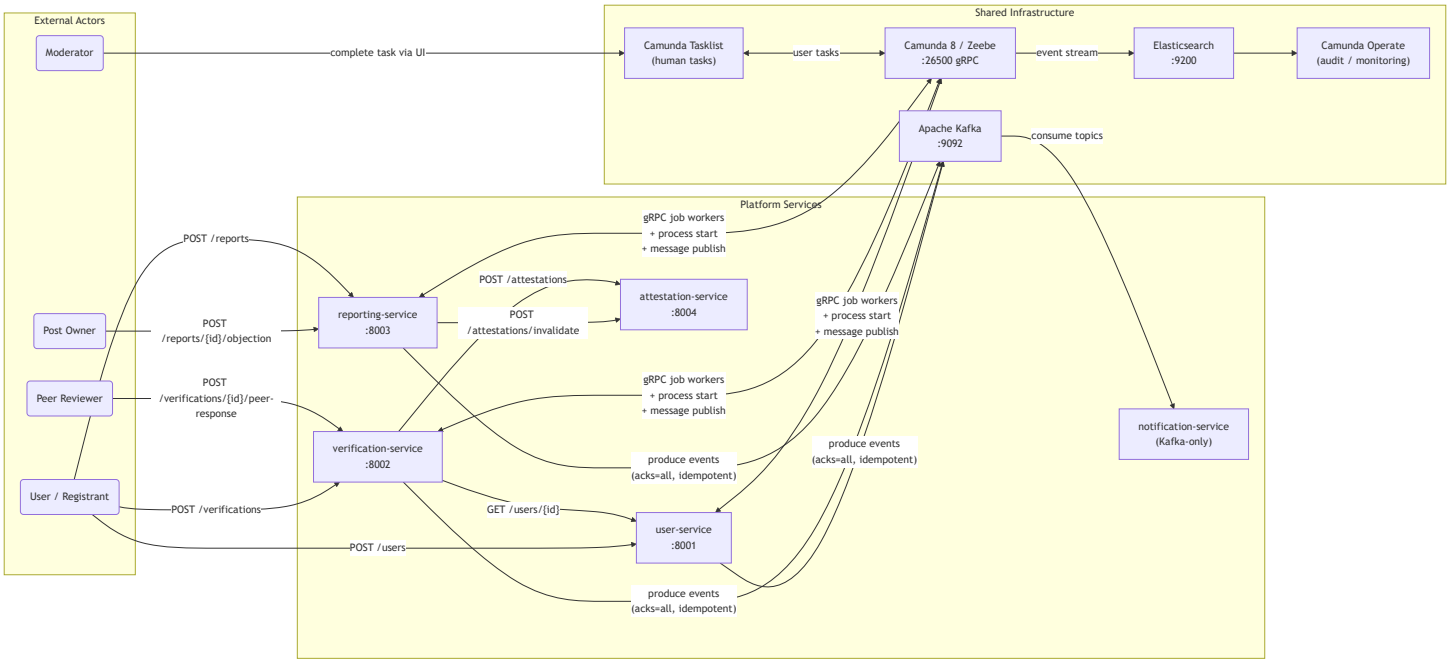
Appendix C - Architecture Diagrams

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

This document provides a set of conceptual-level diagrams covering the full implementation. The diagrams progress from the broad system view down to specific interaction patterns. Each is followed by a short explanation of what it shows and why it matters for the architecture.

1. System Context

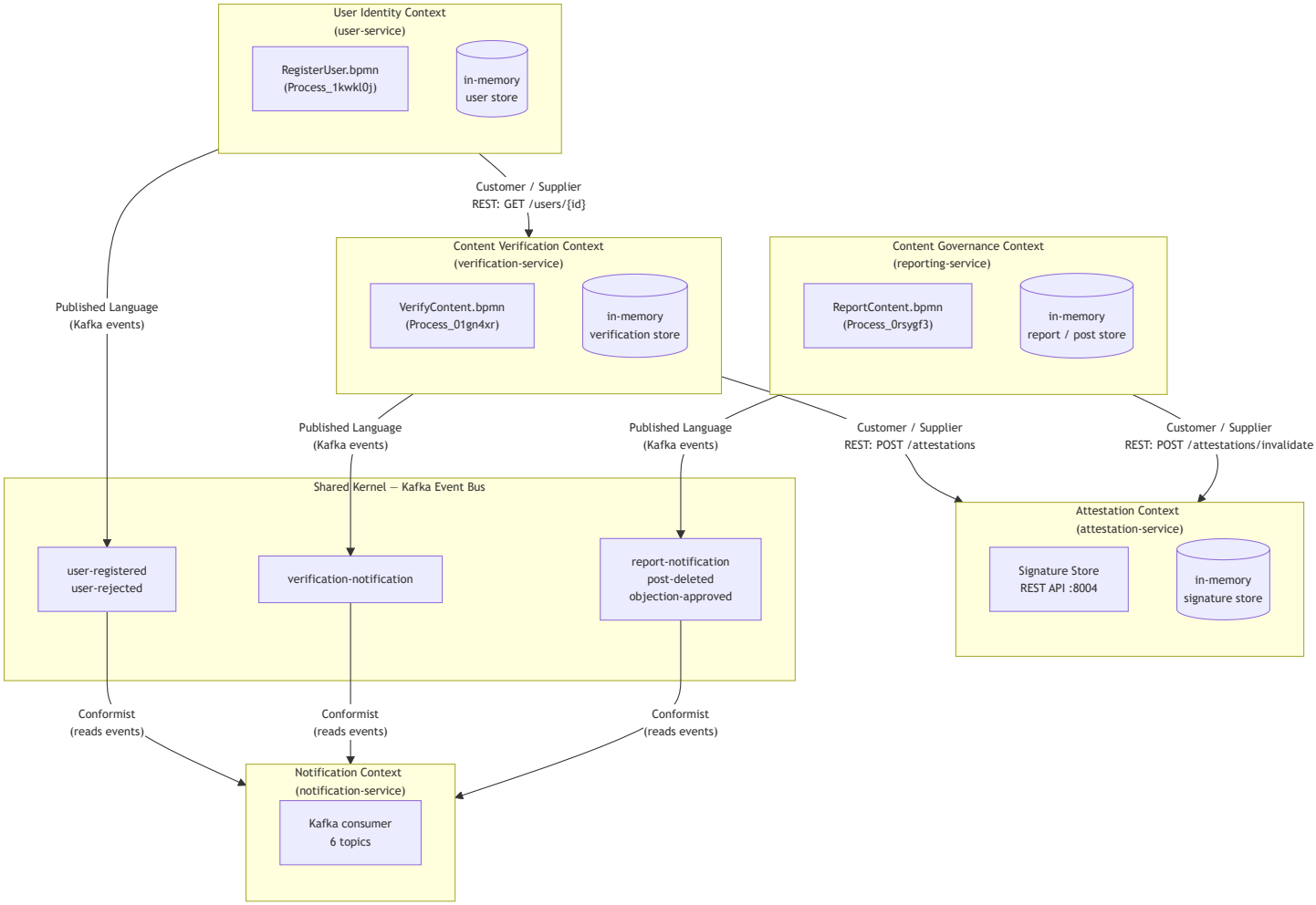
The highest-level view: who interacts with the platform and through which channel. External actors on the left; platform services in the middle; shared infrastructure on the right.



Reading notes: All three orchestrated services (`user-service` , `verification-service` , `reporting-service`) maintain a bidirectional gRPC connection to Zeebe — they start processes, activate jobs, complete jobs, and publish messages over the same connection. The `attestation-service` is the only service with no Zeebe involvement; it is purely a REST-based signature store. The `notification-service` has no HTTP interface at all and only ever reads from Kafka.

2. Context Map (DDD)

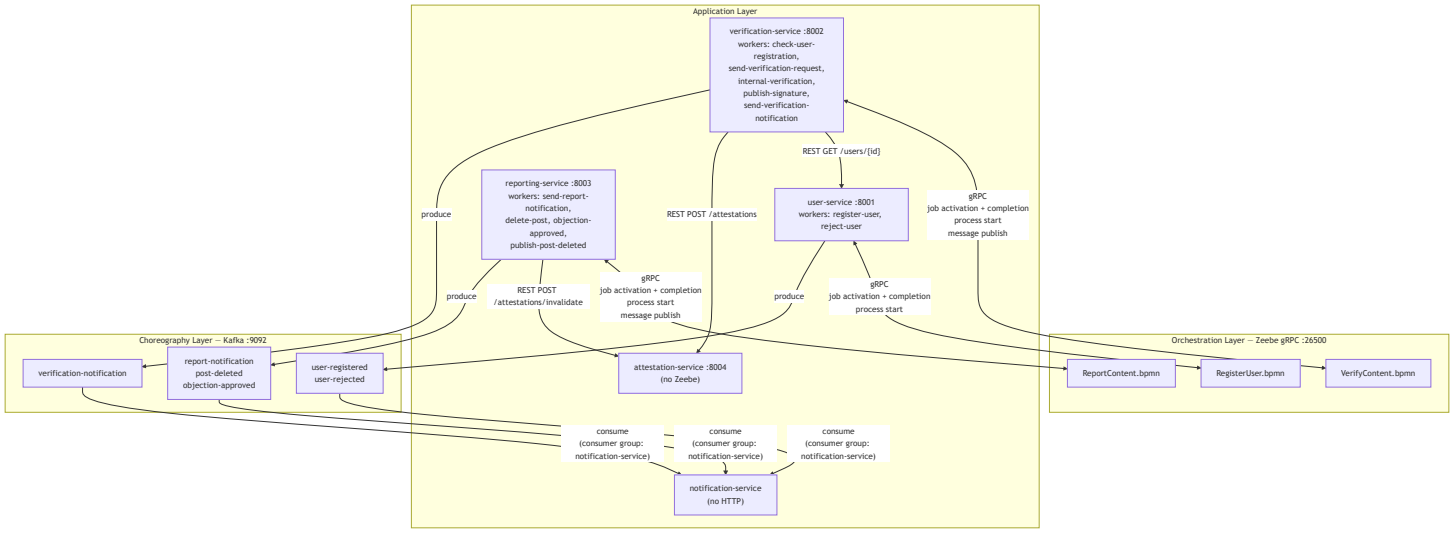
The platform has four bounded contexts. This diagram shows their boundaries and the type of relationship between them, using standard DDD notation.



Reading notes: The `User Identity` context is upstream of `Content Verification` — verifications depend on it to check eligibility, but the user service has no knowledge of verifications. The `Attestation` context is a shared capability used by both `Content Verification` (to store signatures) and `Content Governance` (to invalidate them), but it has no dependency on either. Kafka serves as a shared event bus for loosely coupled notifications following a Published Language pattern; the `notification-service` is a conformist consumer that adapts to whatever events the producing contexts emit.

3. Service Architecture and Communication Channels

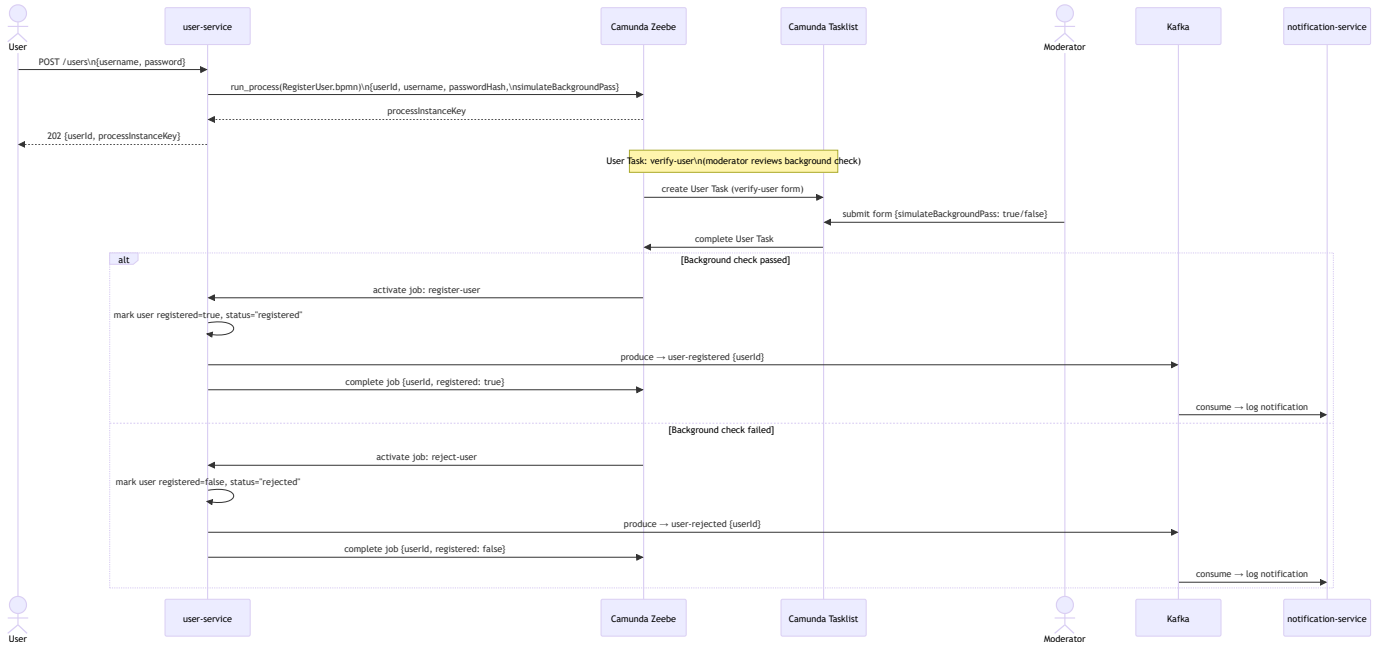
This diagram makes the three different communication channels explicit: Zeebe gRPC (orchestration), synchronous REST (service-to-service calls), and Kafka (choreography).



Reading notes: Each service owns exactly the job types declared in its own BPMN model; there is no cross-service worker. The gRPC connection carries four distinct interaction types: job polling (`ZeebeWorker`), job completion, process instance launch (`ZeebeClient.run_process`), and message publication (`ZeebeClient.publish_message`). REST calls are used only for point-to-point queries where a synchronous answer is needed within a job handler — they are never used across process boundaries for coordination.

4. RegisterUser — Sequence

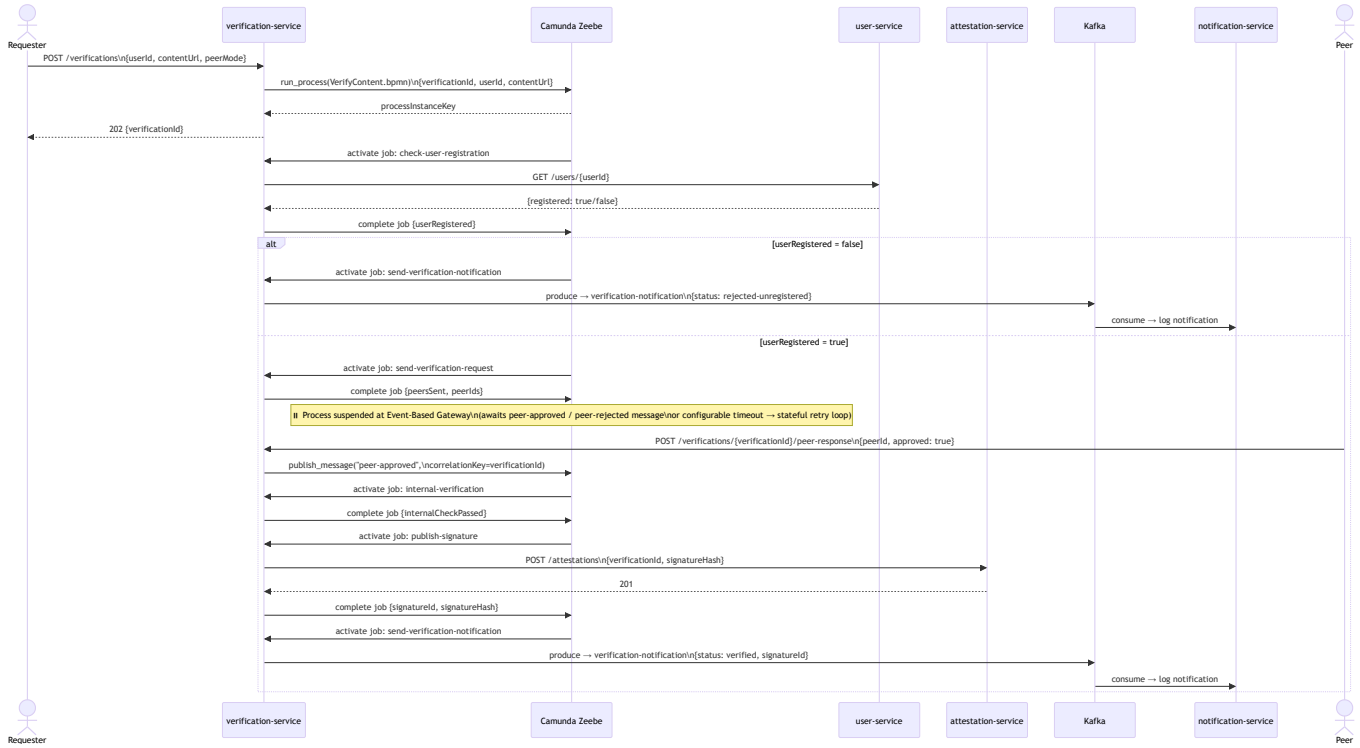
The user onboarding flow. Background verification is modelled as a human task in Tasklist, making the approval decision explicit, auditable, and not automated away.



Reading notes: The `simulateBackgroundPass` flag lets us drive both paths in tests without requiring a real background-check integration. In production this would be replaced by a genuine third-party check result. The `userId` generated at process start is the stable identifier used in all downstream flows — it is the correlation between the identity context and the verification context.

5. VerifyContent — End-to-End Sequence

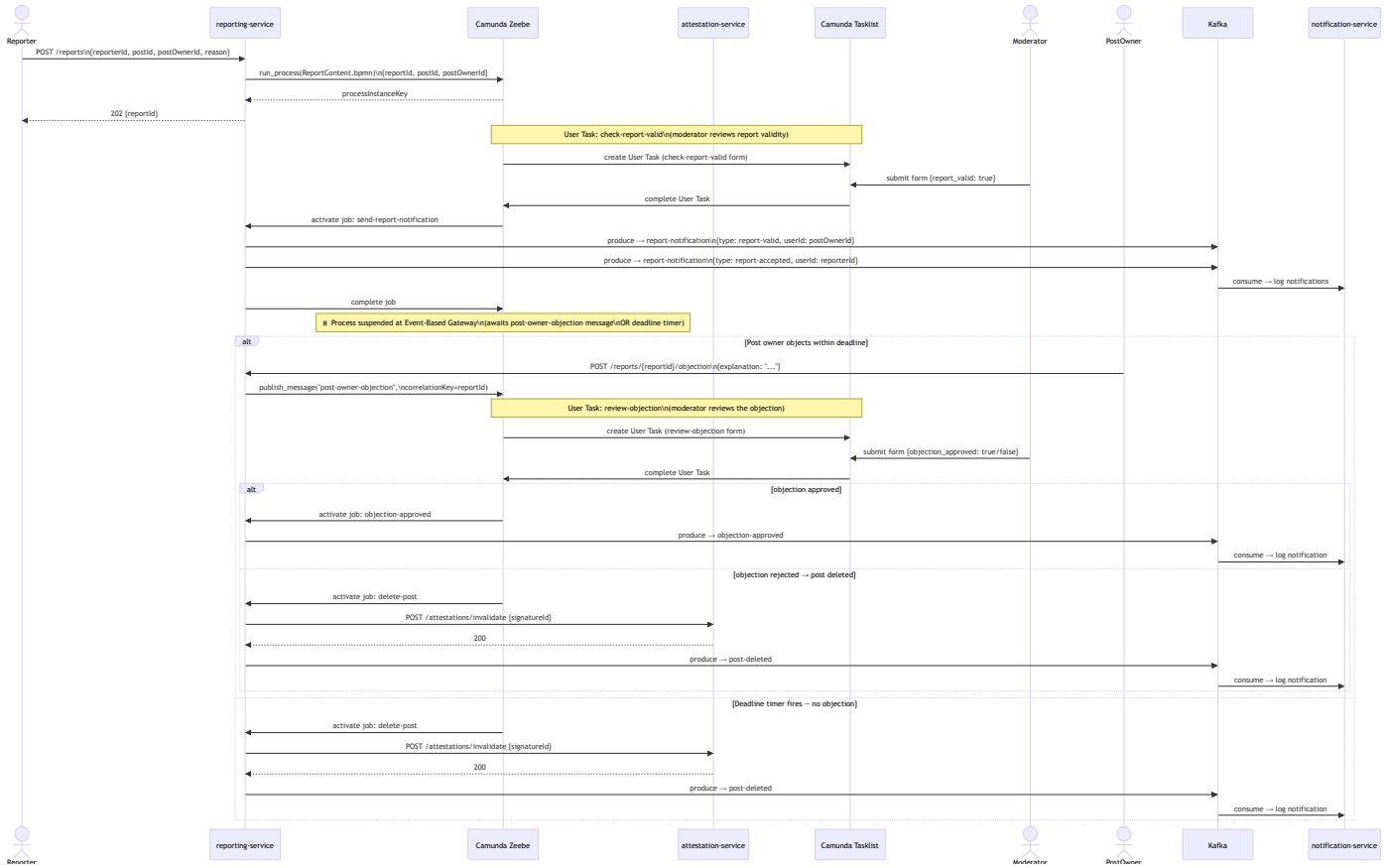
The full lifecycle of a content verification, from API call to Kafka notification. This is the most complex flow and demonstrates orchestration, message correlation, service-to-service REST, and Kafka choreography in a single trace.



Reading notes: The process instance is suspended at the Event-Based Gateway waiting for a peer message or a timeout. During that suspension, zero CPU or memory is consumed by the application — the state lives entirely in Zeebe's event-sourced log. The `verificationId` UUID is both the client-facing identifier (used in the REST URL) and the Zeebe correlation key, which eliminates any secondary routing table.

6. ReportContent — Human Intervention Sequence

The reporting flow demonstrates the human-in-the-loop pattern. A moderator's User Task gates the objection window, and a post owner's REST callback correlates back to the suspended instance.



Reading notes: There are two distinct human tasks in this flow. The first (`check-report-valid`) is always required — no automation replaces the moderator's initial judgement. The second (`review-objection`) is conditional: it only fires if the post owner submits an objection within the deadline. Both User Tasks are visible in Camunda Tasklist with rendered forms, and both completions are recorded in Camunda Operate's audit history.

Summary Table

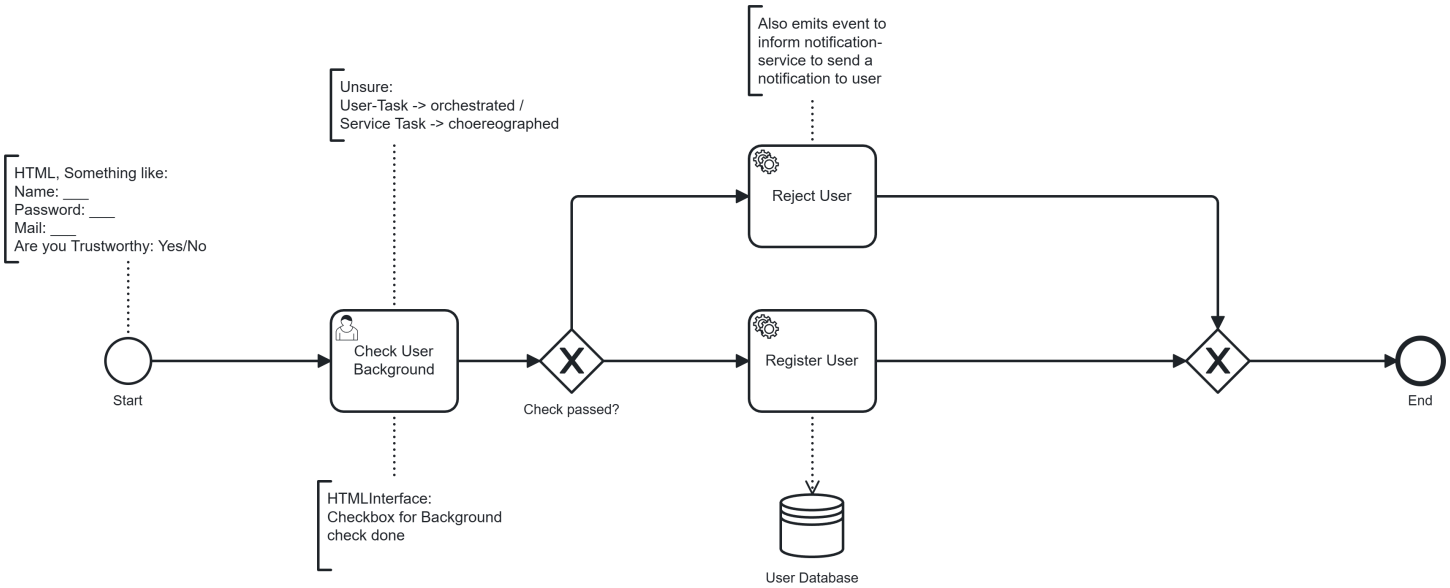
Diagram	Pattern / Concept	ADR Reference
1. System Context	Overall platform boundary and actors	—
2. Context Map	Bounded contexts, upstream/downstream relationships	ADR-0006
3. Service Architecture	Communication channels: gRPC, REST, Kafka	ADR-0001, ADR-0005
4. RegisterUser Sequence	Human task onboarding, Kafka notification	ADR-0003, ADR-0006
5. VerifyContent Sequence	Orchestration, message correlation, Kafka	ADR-0007, ADR-0005
6. ReportContent Sequence	Human intervention, objection correlation, deletion saga	ADR-0003, ADR-0007

Appendix D - BPMN Models and Forms

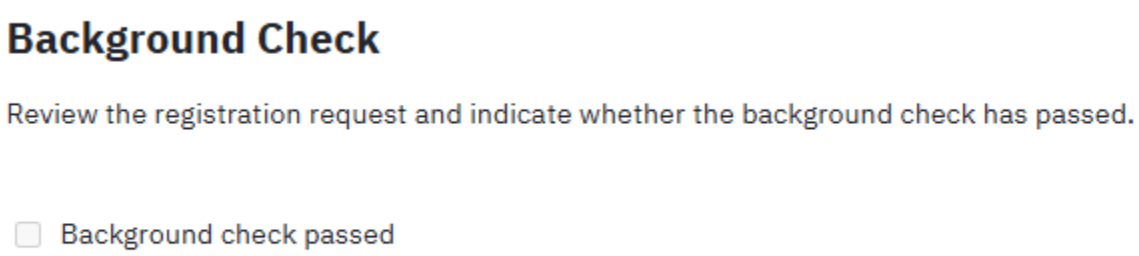
- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

This document provides visual representations of the executable BPMN processes and their associated Camunda forms used in the project.

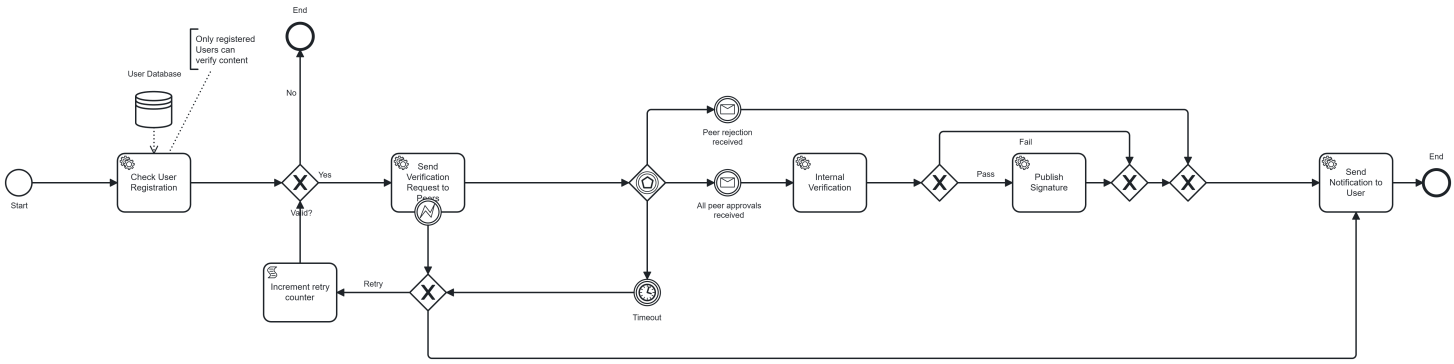
1. RegisterUser Process



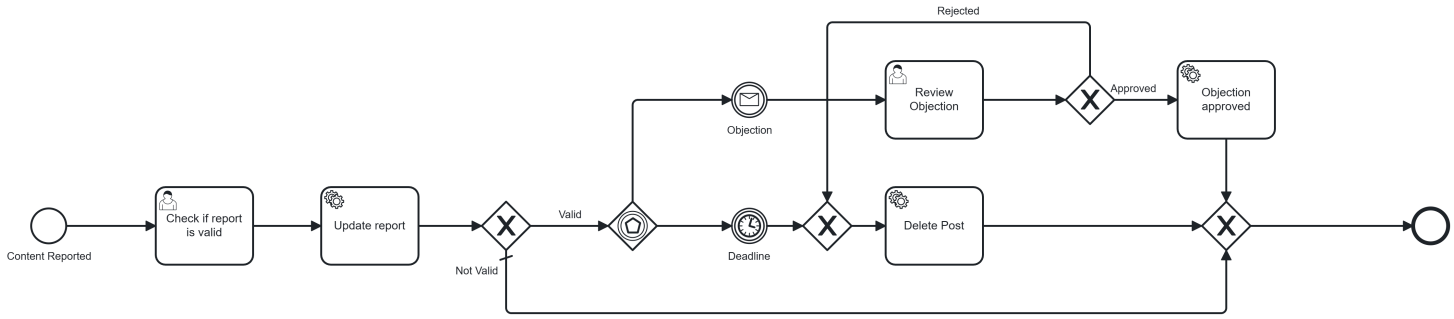
1.1 Verify User Form



2. VerifyContent Process



3. ReportContent Process



3.1 Check Report Validity Form

Check Report Validity

Review the submitted content report and determine whether it describes a genuine policy violation.

- [ ] Report is Valid

3.2 Review Objection Form

Review Objection

The post owner has objected to this report. Review their objection and determine whether it should be rejected (post is deleted) or upheld (post remains).

- [ ] Reject Objection

0001 - Coordination Pattern: Hybrid Orchestration and Choreography

14.04.2026

Status

Accepted

Context

Our platform needs to coordinate work across five services: `user-service`, `verification-service`, `reporting-service`, `attestation-service`, and `notification-service`. These match three distinct business flows: user registration, content verification, and content reporting. Each of these flows has a different character. The content verification flow, for example, involves waiting for an unpredictable number of peer verdicts, enforcing configurable timeouts, making a binary approve-or-reject decision, and potentially issuing a cryptographic signature. These are multi-step, stateful, legally significant outcomes where we need to know at any moment exactly where a given request stands.

The notification flow, on the other hand, is a side-effect. When a verification completes or a post gets deleted, any number of downstream consumers need to be informed, but none of them need to coordinate with each other, and no consumer failing should block the primary flow from completing.

Two established coordination patterns exist for this kind of system. Pure orchestration places a central workflow engine in charge of every interaction — every step, every timeout, every conditional branch is explicitly modelled and driven by the engine. This gives excellent visibility and control but creates a deployment dependency on the engine for every inter-service interaction, including simple notifications. Pure choreography does the opposite: services react to events independently with no central coordinator, which is great for loose coupling but makes the stateful, conditional business flows hard to reason about, hard to observe, and very difficult to compensate correctly when something goes wrong.

The tension is real. Our platform has flows that genuinely need the tight control of orchestration and others where choreography is the more natural and maintainable fit.

Decision Drivers

- Reliability
- Scalability
- Maintainability

Options Considered

- **Option 1**: Pure Orchestration. Pros: Centralized control, easy to monitor. Cons: Tight coupling, single point of failure.
- **Option 2**: Pure Choreography. Pros: Loose coupling, highly scalable. Cons: Hard to monitor, complex error handling.
- **Option 3**: Hybrid Approach (Chosen). Pros: Best of both worlds. Cons: Increased complexity.

Decision

We adopt a hybrid coordination strategy: Camunda 8 (Zeebe) orchestrates all three core business processes `RegisterUser.bpmn`, `VerifyContent.bpmn`, and `ReportContent.bpmn` while Apache Kafka handles the loosely coupled, side-effect notification flows.

The rule for which pattern to apply is straightforward. Any flow with complex conditional branching, asynchronous waits with defined timeouts, legally or reputationally sensitive outcomes, or required compensating actions belongs in a BPMN model under Zeebe's

control. All three of our business processes satisfy these criteria. The content verification flow alone involves a user-registration check, dispatching requests to peers, waiting for their verdicts (with a timeout), running an internal check, calling the attestation service, and finally notifying the requester. Each step represented as an explicit BPMN task with its own retry and failure semantics.

Notifications such as `verification-notification`, `report-notification`, `user-registered`, `user-rejected`, `post-deleted`, and `objection-approved` are published to Kafka topics by the relevant service once the orchestrated step completes. The `notification-service` subscribes to all of these topics independently. This service has no HTTP interface at all; it is a pure Kafka consumer. If it restarts or falls behind, the primary workflows are completely unaffected, and no messages are lost because Kafka retains the log.

Consequences

The hybrid approach means each coordination style is used where it actually fits, rather than being stretched to cover use cases it handles poorly. For the orchestrated flows, every in-flight process instance is durably persisted by Zeebe, observable in Camunda Operate, and recoverable after a worker crash without any application-level persistence code on our part. For the notification flow, new consumers can subscribe to existing Kafka topics at any time without changing anything in the producing services.

The downside is that the team needs to understand both patterns well. Debugging a process that spans both a BPMN engine and a Kafka topic requires familiarity with Zeebe's gRPC API, Camunda Operate, and Kafka consumer group offsets simultaneously. There is also a meaningful operational dependency: Zeebe must be running and reachable for any new process instance to start. The `notification-service`, by contrast, is entirely optional from the perspective of the core flows — a nice property that pure orchestration would not give us.

Positive Consequences

The orchestrated core flows benefit from the centralised control that pure orchestration provides: every BPMN task has explicit retry semantics, every in-flight instance is durably persisted by Zeebe, and every step is fully auditable via Camunda Operate without any additional logging code. At the same time, the Kafka-based notification side retains the loose coupling and scalability that pure choreography offers: the `notification-service` is entirely independent of the services that produce events, can fall behind or restart without affecting core flows, and can be scaled horizontally via consumer group partitioning. New consumers (an analytics service, an email gateway) can subscribe to existing topics at any time without touching existing producers or redeploying any orchestrated service.

Negative Consequences

The main drawback of this implementation is the increased complexity it comes with. The operational surface area is larger than a pure choreography system. The team must understand both Zeebe and Kafka, and failures require diagnosis in two different subsystems. So, there is also a drawback in the maintainability of the implementation.

Implementation Considerations

Any new business flow added to the platform should be assessed against the same criteria we applied here. If it requires stateful waits, defined timeouts, conditional branching, or compensating transactions, it belongs in a BPMN model. If it is a side-effect that should not block the primary flow, Kafka is the right vehicle.

0002 - Workflow Engine Selection: Camunda 8 over Camunda 7

14.04.2026

Status

Accepted

Context

We need a BPMN workflow engine to orchestrate our three core business processes. The two most natural candidates in the Camunda ecosystem are Camunda 7 (the long-established, Spring Boot native platform) and Camunda 8 (the newer, Zeebe-based engine built around an event-sourced, distributed architecture).

Our team writes Python 3.12. None of our services use Java or Spring Boot, and we have no intention of introducing either. This is not a stylistic preference — it is a project constraint, because mixing runtimes would fragment our build pipeline, container definitions, and team knowledge. The client library situation for the two engines is therefore the most important practical factor in the decision.

Camunda 7 was designed around a Java process engine. Its REST API can technically be called from any language, but the available Python clients are thin wrappers that require managing raw HTTP, polling, and serialization by hand. There is no officially supported, idiomatically Python job-worker client for Camunda 7. Running job workers against it from Python means either a bespoke polling loop or a third-party library with no active maintenance.

Camunda 8 has `pyzeebe`, an actively maintained, asyncio-native Python library that provides first-class job-worker support, message publication, and process lifecycle management over gRPC. This is not a workaround — it is the intended integration path for Python services.

The second relevant dimension is state persistence. Camunda 7 stores process state in a relational database (typically PostgreSQL or MySQL). All variable mutations, token movements, and wait states require transactional writes to a shared schema. Under high concurrency this becomes a bottleneck and a single point of failure. Camunda 8's engine (Zeebe) uses an event-sourced log persisted to RocksDB, partitioned across brokers. Process state is the accumulated result of an append-only log; the engine never needs to update rows, only append records.

Decision Drivers

- Interoperability: Python 3.12 is the sole runtime; no Java or Spring Boot dependency may be introduced
- Reliability: process state must survive worker crashes and redeployments without application-level checkpointing
- Scalability: the persistence model must not become a bottleneck under concurrent process instances
- Maintainability: adding new job workers must require minimal boilerplate

Options Considered

Camunda 7 has a mature ecosystem and strong tooling, but its process engine is Java-native. Its REST API can be called from Python, but there is no officially supported, idiomatic Python job-worker client. Running workers from Python requires either a hand-rolled polling loop or an unmaintained third-party library. Process state is stored in a relational database, which becomes a write bottleneck under high concurrency. Rejected on grounds of interoperability and scalability.

Temporal / Conductor both have Python SDKs, but neither executes BPMN natively. Our requirement is to model and run processes as BPMN on a BPMN engine. Rejected on grounds of BPMN incompatibility.

Camunda 8 / Zeebe (chosen) provides `pyzeebe`, an actively maintained asyncio-native Python library with first-class job-worker support over gRPC. Its event-sourced, append-only persistence to RocksDB removes the relational write bottleneck. Process state survives worker crashes by design, with no application-level persistence code required.

Decision

We use Camunda 8 (Zeebe), operated in self-managed mode via `c8run`, as the workflow engine for all three BPMN processes. Our Python services interact with it exclusively through `pyzeebe`, which is used in all three orchestrated services: `user-service`, `verification-service`, and `reporting-service`. Each service runs a `ZeebeWorker` in a dedicated asyncio event loop on a background thread, and uses `ZeebeClient` when it needs to start a process instance or publish a message.

We run Zeebe in single-broker mode locally, which is sufficient for course-project scale. The architecture does not require changes to move to a multi-broker cluster because the `ZEEBE_ADDRESS` environment variable is the only coupling point between the services and the engine.

We considered self-hosted Camunda 7 briefly. The Python integration story was the deciding factor against it. We also looked at lightweight alternatives such as Temporal and Conductor. Both have Python SDKs, but neither supports BPMN natively — our course requirement is to model processes in BPMN and execute them on a BPMN engine, so that ruled them out.

Consequences

Positive Consequences

`pyzeebe` gives our Python services a clean, idiomatic way to implement job workers using simple `async def` functions decorated with `@worker.task(task_type="...")`. Writing a new job worker takes a few lines of code and no knowledge of gRPC internals. Message correlation and process launching follow the same pattern. The event-sourced persistence model means that process variable changes survive worker restarts without any application-level persistence logic. If a service crashes between a Zeebe job activation and its completion, Zeebe retries the job automatically once the retry timeout elapses. We do not need to checkpoint our own state to a database for crash recovery. Horizontal scaling is a configuration change rather than an architectural one.

Negative Consequences

Zeebe requires Java 21 to 23 to run in `c8run` mode, which is a runtime dependency outside our Python ecosystem. Every developer needs a working Java installation to run the full stack locally. Zeebe also consumes more memory than a lightweight message router, and its RocksDB data directory requires some care to manage, particularly around disk space during long-running development sessions. Camunda 7's richer query API and direct SQL access to the process history database are not available to us; for audit and monitoring purposes we rely entirely on Camunda Operate, which is adequate but less flexible than raw SQL queries would be.

0003 - Stateful Resilience and Error Handling

14.04.2026

Status

Accepted

Context

Our platform runs long-lived, asynchronous verification and reporting processes. A `VerifyContent` instance might be suspended for minutes waiting for peer verdicts. A `ReportContent` instance might be waiting for a post owner to submit an objection within a defined window. During that time, any of the following can happen: a service container restarts due to a deployment, Zeebe's gRPC connection temporarily drops, the `user-service` is briefly unresponsive, or a developer's laptop goes to sleep.

The question is what happens to in-flight process instances and job executions when any of these events occur. If we have to design and maintain our own crash-recovery layer, writing process state to a database, checking for abandoned instances on startup, and re-enqueueing jobs, the services become significantly more complex and the risk of subtle bugs in the recovery logic is high.

There is also a second, more specific problem: the `delete-post` task in `ReportContent.bpmn` is irreversible. Once a post is deleted and its attestation signature is invalidated, that cannot be undone automatically. If something goes wrong during or immediately after this step, we need a human to review the situation rather than an automatic retry blindly executing the deletion a second time.

The retry problem is not limited to short-lived technical failures. In `VerifyContent.bpmn`, the critical failure mode is peer non-response across an asynchronous waiting period. This is exactly the kind of retry that cannot be handled safely inside a worker process, because the retry state must survive crashes, redeployments, and long timer waits. The retry counter therefore has to belong to the workflow state itself, not to application memory.

Decision Drivers

- Reliability: process state and retry counters must survive worker crashes and redeployments
- Fault Tolerance: a single service failure must not abort an in-flight verification or report
- Resilience: transient failures must be retried automatically; irreversible actions require human oversight rather than blind automated retry

Options Considered

Application-level checkpointing: each service writes its own in-flight state to a database and reconciles on startup. Pros: no hard dependency on the engine for recovery. Cons: significant extra code per service, high risk of subtle bugs in reconciliation logic, retry counter management must be built and maintained from scratch.

Zeebe's built-in job activation and retry model (chosen): the engine marks jobs as active with a configurable timeout; jobs not acknowledged within that window are automatically re-queued. Pros: zero application-level recovery code, engine-guaranteed retry semantics, crash-safe by design. Cons: Zeebe availability becomes a hard dependency for all process progression.

Human fallback via Camunda Tasklist for irreversible steps (chosen): instead of automated retry for destructive operations, failed tasks are routed to a human moderator via a User Task. Pros: prevents accidental double-execution of irreversible actions, makes

oversight explicit in the BPMN model and auditable in Operate. Cons: requires someone to actively monitor the Tasklist; introduces an operational SLA concern.

Decision

We rely on Zeebe's built-in job activation and retry model as our primary resilience mechanism for transient failures. When a `ZeebeWorker` activates a job, Zeebe marks that job as active and starts a configurable activation timeout. If the worker does not complete or fail the job within that window because the service restarted, threw an unhandled exception, or lost its connection, Zeebe automatically makes the job available for the next poll cycle without any application-level intervention. Our services do not need to write their own retry or recovery logic for this class of failure; the engine handles it.

For retries on transient errors we configure the BPMN service tasks with an explicit retry count. When a worker explicitly fails a job (via a Zeebe job failure response with `retries > 0`), Zeebe decrements the counter and re-activates the job after a back-off delay. In addition, `VerifyContent.bpmn` models a genuine stateful retry loop around peer non-response. After `Send Verification Request to Peers`, an Event-Based Gateway waits for either a peer response message or a timer. On timeout or service error, a Script Task increments the `send_verify_retry` process variable and an exclusive gateway routes either back to retry the peer request or forward to a failure notification once the retry cap is reached. The important point is that `send_verify_retry` is workflow state persisted by Zeebe, so the retry count survives crashes and redeployments even though the wait may last minutes.

We follow the at-least-once approach described in the course material: domain state is committed before the Zeebe job is completed. This accepts the possibility of duplicate processing after recovery, but avoids silently losing a state transition. In practice, this means our resilience strategy depends on persisted workflow state and eventual consistency rather than distributed transactions.

For the irreversible `delete-post` task we take a different approach. The BPMN model routes a failed deletion to a Camunda Tasklist User Task where a human moderator can inspect the state and decide whether to retry or dismiss the case. We also apply the same pattern to the `verify-user` and `check-report-valid` steps, which require human judgement by their nature. All three human tasks have corresponding Camunda Forms deployed alongside the BPMN (`verify-user.form`, `check-report-valid.form`, `review-objection.form`).

We deliberately do not implement application-level state checkpointing in any service. The process variables stored by Zeebe are the authoritative state for in-flight instances. The in-memory dictionaries in `verification-service` and `reporting-service` (`_verifications`, `_reports`) are a convenience layer for serving fast read queries; they are not the source of truth for process progression.

Consequences

Positive Consequences

Services contain no crash-recovery logic. There is no startup reconciliation code, no "check for stuck jobs on boot" logic, and no database schema for persisting retry counters. In-flight verifications and reports survive service restarts without data loss. Transient infrastructure hiccups such as a brief network partition between a service container and Zeebe, a pod restart during a rolling deployment, or a momentary timeout calling `user-service` are handled transparently. A process instance that was mid-execution when a container died will resume at the same task once the container comes back up and reconnects. The peer retry loop in `VerifyContent.bpmn` is genuinely stateful rather than an in-process retry, because the timeout state and the `send_verify_retry` counter are persisted in Zeebe and visible in Camunda Operate. Human fallback for irreversible decisions is built into the process model itself, making the oversight mechanism explicit and auditable.

Negative Consequences

Zeebe is a hard dependency. Its availability directly governs the availability of our core flows. If Zeebe itself is unavailable, no new process instances can start and no in-flight jobs can progress. We accept this as a known single-point-of-failure for the current scale of the project. A production system would run Zeebe in a multi-broker, multi-partition cluster to remove this dependency. The in-memory

service state (`_verifications` , `_reports`) is lost on a service restart, meaning GET endpoints return 404 for recently started instances until the process variables stored in Zeebe are eventually reconciled; this is an accepted limitation at the current project scale. The at-least-once completion strategy also means duplicate processing remains possible after recovery, so worker logic and downstream state transitions must remain idempotent. The use of Camunda Tasklist for human fallback also introduces an operational concern: someone needs to actively monitor the Tasklist and action stuck items. For the course project this is the team. For a real deployment, this would require a substantial amount of human resources and time to handle.

0004 - Distributed Data Consistency: Saga Pattern over Distributed ACID Transactions

14.04.2026

Status

Accepted

Context

Our platform spans five services, each with its own data store. The content verification flow touches at least three of them: `verification-service` validates the requester against `user-service`, issues a signature through `attestation-service`, and notifies downstream consumers via Kafka. The reporting flow adds a fourth party — `attestation-service` again — when it invalidates a signature on post deletion.

In a traditional relational system, changes spanning multiple tables within a single database can be wrapped in an ACID transaction. If any step fails, the whole thing rolls back atomically. In our architecture, the same guarantee is fundamentally unavailable: the services run in separate processes with separate databases, and there is no transaction coordinator that can acquire locks across all of them simultaneously.

The standard solution for this in a distributed system is a Saga — a sequence of local transactions, each committed independently, where a failure at step N triggers compensating transactions to undo steps 1 through N-1. Sagas come in two flavours: choreography-based (each service listens to events and decides what to do next) and orchestration-based (a central coordinator drives the sequence and triggers compensations explicitly).

A shared database approach — where all services write to a common PostgreSQL schema — was considered early and rejected. It would eliminate the distributed consistency problem at the cost of eliminating service independence: a schema change in one domain would require redeployment of every service, and a single database failure would take down the entire platform.

Decision Drivers

- Reliability: partial failures across service boundaries must not leave the system in an undefined state
- Maintainability: services must be independently deployable; shared schemas create tight deployment coupling
- Fault Tolerance: a single database failure must not cascade across the entire platform

Options Considered

Shared database: all services write to a common PostgreSQL schema. Pros: ACID transactions available natively, trivial cross-service queries. Cons: tight schema coupling forces coordinated deployments, a single database failure takes down all services, and independent scaling becomes impractical.

Distributed transactions: a transaction coordinator acquires locks across all service databases before committing. Pros: strong consistency across services. Cons: not practically implementable across independent Python microservices; introduces a coordinator as a new single point of failure; high latency under lock contention.

Saga pattern with Zeebe orchestration (chosen): each service commits its own local transaction; compensating actions for failure scenarios are modelled explicitly as BPMN branches. Pros: no distributed locks, services remain independently deployable and

evolvable, compensation logic is visible in the process model rather than scattered across event handlers. Cons: eventual consistency windows exist between service-local state and the process variables in Zeebe; compensation is semantic rather than physical rollback.

Decision

We enforce a strict database-per-service model and use Zeebe as the orchestration coordinator for our sagas. Each service owns its own data and does not write to any other service's store. Cross-service interactions happen exclusively through Zeebe job completions (for orchestrated steps) and Kafka events (for notifications).

Our saga variants follow Camunda's own classification. The `VerifyContent` flow resembles an **Epic Saga**: it is a long-running orchestration with a strict sequence of steps and a defined set of compensating paths if early steps succeed but later ones fail. If the `publish-signature` call to `attestation-service` fails after peer approval, the `send-verification-notification` task still runs — it just reports the appropriate failure status to the user rather than leaving the process in an undefined state.

The `ReportContent` flow is similar in structure but includes an explicit compensating step on the happy path. When a post is deleted, `delete-post` calls the attestation service to invalidate the associated signature. If that call fails, the post is still marked deleted locally and the outcome is logged — the business rule is that the deletion takes precedence, and reconciliation of the signature store is a background concern.

We do not implement a rollback for user registration because it is a one-way onboarding flow; a rejected user is simply never activated, which is a natural terminal state rather than a compensation.

Consequences

Positive Consequences

Services are deployable and scalable independently. Schema evolution in one service does not require coordinated changes across the system. The Zeebe-orchestrated saga gives us a significant advantage over a choreography-based saga for compensations: the process model explicitly models every branch, including failure paths, so compensating logic is visible in the BPMN diagram and not scattered across event handlers in multiple services. Any developer can open `VerifyContent.bpmn` in Camunda Modeler and see exactly what happens in each failure scenario.

Negative Consequences

There are no cross-service foreign key constraints and no joins across service boundaries. Queries that would be trivial in a shared schema — "give me all verifications for a user along with their registration status" — require multiple API calls and application-level aggregation. Temporary inconsistencies between service-local state and the Zeebe process variables are possible after a service restart. For example, `attestation-service` may store a new signature before `verification-service` updates its in-memory record to reflect `"status": "verified"`. There is no distributed rollback — compensation is semantic, not physical — so careful design of compensating tasks is required for any new flow added to the system.

0005 - Message Broker Selection: Apache Kafka for Choreography Events

14.04.2026

Status

Accepted

Context

As described in ADR-0001, we use choreography for notification-style side-effects: events like `verification-notification`, `report-notification`, `user-registered`, `user-rejected`, `post-deleted`, and `objection-approved`. These events are fire-and-forget from the producer's perspective — the `verification-service` publishes a `verification-notification` event after a verification outcome is determined, and it neither knows nor cares how many consumers receive it.

We need a broker that reliably delivers these events, can handle situations where a consumer is temporarily unavailable (without the producer having to retry), and allows us to add new consumers in the future without touching existing producers or reconfiguring the broker topology.

The two broad categories we considered were traditional message queues — represented by RabbitMQ — and log-based streaming platforms, represented by Apache Kafka.

Traditional MQ systems like RabbitMQ use a push model where the broker routes messages to registered consumers and removes them once acknowledged. They are easy to operate and well-suited to task queues. However, they are less well-suited to the pub/sub, multi-consumer fanout we need: adding a second consumer to a RabbitMQ queue means either duplicating the queue (and updating the producer to publish to both) or using an exchange — either way, there is coordination required when adding consumers. Message retention is also bounded; a message not consumed within a TTL is gone.

Kafka uses a pull model built on an append-only, partitioned log. Consumers maintain their own offsets and read at their own pace. Adding a new consumer group changes nothing for existing producers or existing consumers. Messages are retained for a configurable period regardless of whether any consumer has read them, which provides a natural buffer against consumer downtime and also makes replaying past events straightforward.

Decision Drivers

- Extensibility: new consumers must be addable without modifying existing producers or reconfiguring the broker
- Reliability: messages must survive consumer downtime without requiring producer-side retry logic
- Scalability: the broker must support horizontal scaling of consumers via partitioning
- Elasticity: individual consumers must be independently scalable

Options Considered

Direct HTTP calls from producer to each consumer: no broker required. Pros: simple, easy to reason about. Cons: producers must know all consumers; adding a new consumer requires changing producer code; no buffering for consumer downtime.

Traditional MQ (RabbitMQ): push-based, broker routes messages to registered consumers. Pros: easy to operate, low memory overhead. Cons: adding a new consumer requires binding new queues or exchanges, a configuration change that must be coordinated

with the existing deployment; messages are dropped after their TTL if unconsumed.

Apache Kafka (chosen): append-only, partitioned log with consumer-managed offsets. Pros: consumers are fully independent of each other and of producers, configurable message retention survives consumer downtime, horizontal scaling via consumer groups, idempotent producer configuration prevents duplicates. Cons: significantly heavier to operate locally than a simple queue.

Decision

We use Apache Kafka (exposed at port `9092` via Docker Compose, internal broker address `kafka:29092`) as the message broker for all choreography-style event flows. Producers are configured with `acks=all` and `enable.idempotence=True` to avoid duplicate or lost messages under retries. The `notification-service` is a Kafka consumer using a named consumer group (`notification-service`) with manual offset commit after successful message processing.

The `verification-service` and `reporting-service` each create a `confluent-kafka` `Producer` instance at startup (lazily, on first use). The `notification-service` creates a `Consumer` that subscribes to all six notification topics in a single call. Kafka's partition-based routing means the `notification-service` can be scaled horizontally — multiple instances can share the same consumer group and Kafka will distribute the partitions between them automatically.

We deliberately chose not to use RabbitMQ or another traditional MQ. The main reason is the consumer-independence property. We expect to add more consumers to these topics as the platform grows (an analytics service, a billing service, an email gateway). With Kafka, this is a matter of deploying a new consumer with its own group ID and subscribing to the relevant topics. With RabbitMQ the same expansion would require binding new queues to existing exchanges, a configuration change that would need to be coordinated with the existing deployment.

Consequences

Positive Consequences

The `notification-service` is entirely decoupled from the services that produce events. A crash in the notification service does not affect the `verification-service` or `reporting-service` at all; they produce their events to the log and continue. When the notification service restarts, it reads from its last committed offset and processes any messages it missed during the outage. No messages are dropped. New consumers can subscribe to existing topics without any changes to producers or broker configuration. Messages are retained through consumer downtime. The `notification-service` is horizontally scalable via consumer group partitioning. Idempotent producers prevent duplicate events under retry conditions. Kafka's log retention also gives us a short-term audit trail of notification events, viewable through the Kafka UI at port `8079` , which is useful during development.

Negative Consequences

Kafka is significantly heavier than RabbitMQ to run locally, and its configuration surface (broker settings, topic creation, consumer group behaviour) requires more upfront understanding. For the current scale of five services and six topics with moderate throughput, Kafka is somewhat over-engineered. The `acks=all` producer configuration means each produce call waits for all in-sync replicas to acknowledge the write. In a single-broker development setup this effectively means waiting for the broker, which adds a small amount of latency to each event publication. Schema management across topics (currently informal JSON) could also become a maintenance concern as the topic count grows, and debugging consumer lag and offset issues requires familiarity with Kafka tooling.

0006 - Process Model Boundaries: One BPMN Model per Bounded Context

14.04.2026

Status

Accepted

Context

When Camunda is first introduced to a system, there is a tempting but dangerous pattern: create one large BPMN process model that handles everything. Start with user registration, then extend the same model to cover content verification, then add reporting as a sub-process. The result is what practitioners call a process monolith — a single workflow definition that becomes a coordination point for the entire application.

The problem with a process monolith is the same as with any monolith. A change to the peer voting logic in the verification flow requires redeploying the same model that also governs user onboarding. A bug in the report objection timer blocks a hotfix to the registration background check. Teams that own different domains are forced to coordinate releases around a shared artifact. Worse, a high volume of content verification instances inflates the Zeebe partition load for all processes, including the low-volume, low-urgency user registrations.

Our platform has three domains that are conceptually independent: user identity (registration and onboarding), content trust (verification, peer voting, signature issuance), and content governance (reporting, objection, moderation, deletion). They share some data — `userId` appears in both verification and registration — but their lifecycles are completely separate. A user registration can complete without any content verification ever being started. A report can be filed against a post whose original verification process completed months ago.

Decision Drivers

- Maintainability: a change to one process model must not require redeployment of unrelated services
- Scalability: load from one domain must not inflate partition or resource usage for another
- Elasticity: each service and its workers must be scalable independently of the others

Options Considered

Single unified BPMN model (process monolith): all three flows modelled as one process definition or as sub-processes of one. Pros: single deployment artifact, shared process variables across flows. Cons: any change requires redeploying the whole model; high-volume verification instances inflate Zeebe partition load for all flows including low-volume registrations; teams across domains must coordinate every release around one shared artifact.

One BPMN model per bounded context (chosen): each service owns, deploys, and operates its own process definition. Pros: fully independent deployment cycles, domain-isolated partition load, teams can evolve their model without any cross-domain coordination. Cons: cross-domain data correlation (e.g. linking a report back to its original verification) must be handled by convention in application code rather than shared process state.

Decision

We define three independent BPMN process models, each owned and deployed by its corresponding service:

1. `RegisterUser.bpmn` (`Process_1kwkl0j`) is owned and driven by `user-service` . It handles background-checked onboarding and publishes `user-registered` or `user-rejected` events to Kafka when it concludes.

2. `VerifyContent.bpmn` (`Process_01gn4xr`) is owned and driven by `verification-service` . It handles the full peer verification lifecycle, including user eligibility checks, peer distribution, verdict collection, internal review, attestation, and outcome notification.
3. `ReportContent.bpmn` (`Process_0rsygf3`) is owned and driven by `reporting-service` . It covers moderation review, owner notification and objection window, and the conditional deletion path with signature invalidation.

Each service registers its own `ZeebeWorker` that handles only the task types declared in its own BPMN model. There is no cross-service worker — `verification-service` does not handle any task declared in `ReportContent.bpmn` , and vice versa. The process IDs are stable constants in each service (`VERIFY_CONTENT_PROCESS_ID` , `REPORT_CONTENT_PROCESS_ID`) and are never shared across service boundaries.

The `attestation-service` is intentionally excluded from this model. It is not orchestrated by Camunda at all — it provides a REST API for signature storage and invalidation that the orchestrated services call as part of their own task handlers. It is an infrastructure service, not a domain process.

Consequences

Positive Consequences

Services and their process models are independently deployable and scalable. Deploying a change to `VerifyContent.bpmn` is an operation performed only by the team responsible for `verification-service` ; it requires no coordination with the user or reporting teams. Load from one domain does not affect the operational performance of another. Zeebe handles concurrent instances of different process definitions independently, so scaling `verification-service` horizontally to handle a spike in verification requests does not affect the instance throughput of `RegisterUser` processes at all. The independence also simplifies monitoring. In Camunda Operate, filtering by process definition gives a clean view of just the instances relevant to one domain. Diagnosing a stuck verification does not require sifting through registration or report instances.

Negative Consequences

Certain cross-cutting concerns must be handled by convention rather than shared infrastructure. The `userId` that appears in both `RegisterUser.bpmn` and `VerifyContent.bpmn` is not enforced by a shared domain object; `verification-service` validates it with a REST call to `user-service` at the start of the process. Similarly, the correlation between a `reportId` in `ReportContent.bpmn` and the original verification that produced the post's signature is maintained by the `reporting-service` locally, not by a shared process link. Cross-domain queries such as "show me the full lifecycle of a piece of content from submission to deletion" require aggregating data from multiple services and process definitions, because there is no built-in join in Zeebe or Camunda Operate across process definitions.

0007 - Asynchronous Message Correlation via Zeebe

14.04.2026

Status

Accepted

Context

Two of our three BPMN flows require the process engine to wait for external signals that arrive at unpredictable times. In `VerifyContent.bpmn`, a process instance is suspended at a message catch event after peer requests are dispatched. It waits there until a peer submits a verdict via `POST /verifications/{verificationId}/peer-response`. The process does not know in advance how long that will take — it might be seconds in an automated test or hours in a real scenario. In `ReportContent.bpmn`, a process instance waits for the post owner to optionally file an objection via `POST /reports/{reportId}/objection` within a defined deadline.

The core challenge is routing: when a peer verdict arrives at the `verification-service`, which specific process instance does it belong to? There could be hundreds of concurrent verifications in flight. Similarly, an objection arriving at the `reporting-service` must find its specific report process, not any other one.

The naive alternative is to implement this with synchronous blocking calls. The REST handler that receives the peer verdict could look up the process instance key and call the Zeebe REST API synchronously to complete the waiting receive task. This would work for simple cases but creates a tight coupling between the HTTP layer and the engine, requires the handler to manage the activation lifecycle itself, and would struggle with exactly-once semantics if the REST call is retried.

Another alternative is to have the process poll for verdicts on a timer. The service could implement a periodic task that checks a database for pending verdicts and resumes the process accordingly. This moves the correlation logic out of the engine and into application code, increasing complexity and creating a polling delay between verdict submission and process resumption.

Decision Drivers

- Performance: peer verdict and objection callbacks must reach their waiting process instance with low latency, without polling delays
- Reliability: a message published before the target instance reaches its wait state must not be silently lost
- Scalability: thousands of concurrently suspended instances must be supportable at minimal active resource cost

Options Considered

Synchronous blocking REST call: the callback handler looks up the process instance key and calls Zeebe's REST API directly to complete the waiting receive task. Pros: simple, one call per callback. Cons: tightly couples the HTTP layer to engine internals; fragile under client retries without idempotency guards; the handler must manage job activation lifecycle itself.

Application-level polling: a periodic background task checks a database for pending verdicts and resumes the process accordingly. Pros: decouples the HTTP handler from the engine. Cons: polling delay between verdict submission and process resumption; adds a database dependency; correlation logic moves out of the process model and into application code.

Zeebe native message correlation (chosen): the callback handler publishes a named Zeebe message with a correlation key; Zeebe routes it internally to the matching suspended instance. Pros: no polling, no secondary routing tables, message is buffered by the engine if the instance has not yet reached its wait state, wait state is visible in Camunda Operate. Cons: messages with no matching subscriber are silently dropped after the configured TTL.

Decision

We use Zeebe's native message correlation mechanism for all asynchronous callbacks. When an external signal arrives — a peer verdict or an owner objection — the receiving REST endpoint publishes a named Zeebe message with a correlation key that uniquely identifies the target process instance. Zeebe routes the message to the correct suspended instance and delivers the associated variables into the process scope.

For peer verdicts in `verification-service`, the correlation key is the `verificationId` (a UUID generated at process start and passed as an initial process variable). The message name is either `peer-approved` or `peer-rejected`, depending on the accumulated vote tally. The `_publish_camunda_message` function calls `ZeebeClient.publish_message(name, correlation_key, variables)` using the shared asyncio event loop.

For owner objections in `reporting-service`, the correlation key is the `reportId`. The message name is `post-owner-objection`. The objection REST handler calls `ZeebeClient.publish_message` with the explanation payload and the `reportId` as the correlation key.

In both cases, the BPMN process models have corresponding message catch events subscribed to these message names, with the correlation key expression referencing the appropriate process variable (`= verificationId` or `= reportId`). Zeebe enforces that a message is delivered to exactly one instance — if no instance is waiting for a given correlation key, the message is buffered for a configurable TTL and then discarded.

Consequences

Positive Consequences

Process instances that are waiting for external signals do not poll. They are fully suspended in Zeebe, consuming no CPU, no memory in the application tier, and no database connections until the correlated message arrives. This is important for scalability: thousands of verifications can be simultaneously suspended with zero active resource consumption. The `verificationId` and `reportId` are stable, client-visible identifiers that peers and post owners use in their callback requests. Because they are also the Zeebe correlation keys, there is no secondary mapping table required to connect an incoming REST request to its process instance. The routing is done entirely inside the engine. The BPMN models accurately represent the wait semantics as message catch events, which are visible in Camunda Operate. An engineer looking at a stuck verification instance can immediately see that it is suspended at "Waiting for Peer Verdict" and that the expected message has not arrived.

Negative Consequences

Correlation keys must be unique per active message subscription. If two concurrent processes subscribe to the same message name with the same correlation key, the behaviour is undefined. Our UUIDs make this collision extremely unlikely, but it is still a design constraint for any future correlated flow. Messages published without a matching subscriber are also silently dropped after the TTL. If a peer submits a verdict after the verification process has already timed out and completed, the message finds no subscriber and is discarded. This is the correct behaviour, but callers receive no error indicating the verdict was ignored. If needed, we would have to implement a response check at the application layer.

0008 - Auditing and Read Models: Camunda Operate as a Separated CQRS Read Side

14.04.2026

Status

Accepted

Context

Our platform makes legally significant decisions: content is certified as trustworthy, posts are permanently deleted, users are approved or rejected based on background checks. For a system with these kinds of outcomes, auditability is not optional. Regulators, platform operators, and the users themselves have a legitimate interest in being able to ask: "What happened to this verification? Who approved it? When did the timeout fire? What was the state of the process when the deletion was triggered?"

The write side of our system — Zeebe — is an event-sourced engine. Every change to a process instance is recorded as an immutable event in an append-only log and snapshotted to RocksDB. This is excellent for durability and crash recovery, but it is a poor fit for rich, ad-hoc audit queries. Reading the history of a process instance from RocksDB requires either a Zeebe gRPC query or internal engine APIs. Neither is designed for efficient filtering, pagination, or cross-instance aggregation. Running audit queries against the execution engine also risks interfering with its performance under load.

The classic solution to this tension is CQRS (Command Query Responsibility Segregation): separate the model used to execute commands from the model used to answer queries. The execution engine is optimised for the write path; a separate read model is built from the same event log and optimised for queries.

Implementing our own CQRS read model would mean writing an event consumer that subscribes to the Zeebe event stream, transforming each event into a queryable representation, and maintaining that store. This is a significant engineering effort for a project that already has a broad scope.

Decision Drivers

- Auditability: every step of every process must be traceable after the fact, including variable state, retries, and human task completions
- Performance: audit queries must not compete with execution engine throughput
- Maintainability: the audit infrastructure must require no bespoke development or ongoing schema maintenance

Options Considered

Custom CQRS read model: write an event consumer that subscribes to the Zeebe export stream and projects events into a queryable store such as PostgreSQL. Pros: full control over the query schema, flexible reporting. Cons: significant engineering effort, a new service to deploy and maintain, schema must evolve in lockstep with BPMN model changes.

Direct Zeebe gRPC queries for audit: query process history via `ZeebeClient` API calls at query time. Pros: no additional infrastructure. Cons: Zeebe's gRPC API is not designed for ad-hoc filtering, pagination, or cross-instance aggregation; audit query load competes directly with execution engine throughput.

Camunda Operate as read model (chosen): Operate subscribes to the Zeebe event stream and maintains a continuously updated Elasticsearch-backed projection. Pros: zero custom code, rich filtering and pagination out of the box, query load is fully isolated from the execution engine, bundled with Camunda 8 Run at no extra setup cost. Cons: Elasticsearch adds meaningful memory overhead to the local stack; the read model is eventually consistent with a sub-second lag.

Decision

We use Camunda Operate as the read model for all process audit and monitoring needs. Operate is bundled with Camunda 8 Run and runs as part of the same `c8run` deployment. Internally, Operate maintains a continuously updated projection of process instance state in Elasticsearch. It subscribes to the Zeebe event stream and indexes every process instance lifecycle event, variable change, job activation, message correlation, and error into Elasticsearch documents. This gives it fast, filterable, paginated access to the full history of every instance across all three process definitions.

The separation between Zeebe's execution engine and Operate's Elasticsearch-backed read model is precisely the CQRS split we need without having to build it ourselves. Queries to Operate's UI or REST API do not touch the Zeebe execution engine at all — they go directly to Elasticsearch. A spike in audit queries does not degrade process execution throughput.

For the specific accountability requirements of our platform, Operate provides everything we need: per-instance history with timestamps, variable state at each step, job retries and incident records, and message correlation events. An operator can open any process instance and see the exact sequence of events that led to its current state.

Camunda Tasklist serves a complementary role on the human task side. When a moderator action is required — reviewing a user background check via `verify-user.form`, validating a report via `check-report-valid.form`, or reviewing an owner objection via `review-objection.form` — the task and its associated data are visible and actionable in Tasklist. Completed human tasks are also recorded in Operate's audit history.

We do not build a custom event consumer or a separate audit database. The operational cost of maintaining a bespoke event projection is not justified given that Operate already provides the capability.

Consequences

Positive Consequences

Every step of a content verification, from the initial `check-user-registration` job through peer verdict correlation and attestation signature issuance, is traceable in Camunda Operate without any additional code. The same is true for the reporting and user registration flows. This satisfies our auditability requirement with zero application-level audit logging code. Full process instance history is available from the moment Camunda 8 Run starts. Audit queries are served from Elasticsearch, isolating query load from execution throughput. Human task completions are recorded as part of the same audit history. No custom event consumers or audit schemas need to be maintained.

Negative Consequences

Operate's dependency on Elasticsearch means that a full local stack requires Elasticsearch to be running, which is part of `c8run` but adds memory overhead. In the default `c8run` configuration Elasticsearch listens on ports `9200` and `9300`, which can be a friction point on developer laptops with limited RAM. The Operate read model is eventually consistent with the Zeebe execution engine. There is a short lag, typically sub-second, between a process event occurring in Zeebe and that event appearing in Operate's Elasticsearch index. This is acceptable for audit and monitoring purposes, but the execution engine remains the authoritative source and Operate is only a projection of it. Custom audit reports that require joins across process definitions, or that need to correlate Zeebe history with data from application service databases, cannot be expressed in Operate's UI. For those cases, direct Elasticsearch queries against Operate's indices are possible but constitute an undocumented integration contract.